

A novel methodology for portfolio construction and a comprehensive deep learning framework for devising trading and investment strategies

Anand Venkataraman*

CQF Final Project Report for Jan 2023 Cohort (Submitted on Aug 2023)

1 Abstract

In the ever-changing world of financial markets, this research introduces a forward-thinking and pioneering approach to building portfolios and developing investment and trading strategies. This study blends innovation with practicality by proposing a new way to build portfolios and predict both stock trends and value.

First, a fresh method comes to light for choosing stocks from the famous SP500 indices. It uses a clever score that combines how much to invest in each stock, the PEG ratio, and how diverse stocks are across different sectors. This method is applied to the top 25 SP500 stocks, and from them, a portfolio of 10 standout stocks emerges. Impressively, this portfolio outperforms all others, including the index itself, confirming the effectiveness of our approach.

A new era of possibilities opens with a comprehensive deep learning framework, ready to predict stock price trends accurately. It focuses on volatility spreads, using a simple dataset containing only SP500 prices and VIX (Implied Volatility). By fine-tuning the parameters, the power of Long Short-Term Memory (LSTM) networks is harnessed, delivering an accuracy of around 85-90 %, while base classifier (random forest) delivered an accuracy of 93 %. Interestingly, while simpler datasets and conventional classifiers also show strength, LSTM models shine when it comes to predicting stock values (consistent accuracy of accuracy of 99 %) and volatility index (accuracy of 86 %). Interesting insights on popular hyperparameter tuning algorithms, has also been provided.

Beyond predicting trends, this research reveals the interconnectedness of volatility and market sentiments. This collection of insights, combined with a forward-looking view on trading strategies, creates a bridge between predictive abilities and practical decision-making.

This study is comprehensive and innovative. It could chart a new path in portfolio construction and strategy development. With achievements in stock selection, deep learning, and sentiment analysis, this work would be a guide for devising investment and trading strategies.

*Corresponding author (email) : v.anand@barclays.com

2 Introduction

Stock prices fluctuate due to a complex interplay of uncertain factors, including global economic data, unemployment rates, country monetary policies, and more. Stakeholders aim for profits through thorough market analysis. Challenges arise in creating reliable prediction models due to data complexity and stock markets' inherent chaotic nature. Feature selection from financial data adds complexity wherein some prefer technical indicators, others historical data. Balanced feature selection is key, as over-inclusion complicates models and under-inclusion limits accuracy. Collinearity among variables also affects performance. Optimal models yield accurate predictions and insights. Research has explored variable-stock behavior links, offering mixed model accuracy. Variable quality influences success, emphasizing balanced feature combinations.

Finance's quantitative analysis history spans basic to complex models. Not all gain acceptance. Early ARIMA model focused on price and volume, struggling with non-stationary data. Classical models assume linearity, challenging real-world data's nonlinearity. The eighties saw more interest due to data analysis advancements. Spreadsheets emerged, data collection automated, and computing power improved, aiding predictive models. With vast data, tech growth, and classical time series model challenges, researchers embraced artificial neural networks and deep learning for sequential data modeling. These methods capture complex relationships and efficiently extract key insights. Various deep learning architectures handle diverse problems and data structures. Basic feedforward neural networks lack sequential data understanding, as they process inputs independently. Recurrent neural networks (RNN) address this by looping information over time. RNNs suit sequential data modeling like stock predictions, translations, and signal processing. Training involves minimizing prediction errors via parameter updates using gradients, called backpropagation. However, vanishing gradient issues can arise, especially in early network layers [Hochreiter, 1998]. LSTM, a recurrent neural network, tackles the vanishing gradient problem by preserving information over longer periods.

In summary, extensive research has delved into stock market prediction. Some studies emphasize intricate statistical or machine learning methods, overlooking the significance of underlying variables. Others solely rely on fundamental data, neglecting potential influential factors. There are also models that combines market variables and maintains architectural simplicity, while other models that have dealt extensively with large datasets (with extensive features). However, these models have looked at the 'stock price' problem, either from a trend (classification) or a value (prediction) perspective, on a discrete basis.

There seems to be a demand for a comprehensive framework that would suit for both trend prediction and price prediction, on a holistic basis. *Our contribution involves crafting a framework, that strikes this balance, capturing stock market behavior using volatility spreads using a simplistic dataset (to avoid complexities involving features). Our work focusses on model building and hence there is extensive focus on aspects hyperparameter tuning in our work.*

Stock Selection Methodology Numerous researchers have explored stock selection methods. For instance, Lin et al. [2022] performed stock selection based on a special ratio which is forward PE divided by trailing PE. They claim that this ratio can better evaluate the growth of individual stocks. Truong et. al [2009] discovered low PE stocks outperform high PE stocks. Easton et al. [2004] utilized the PEG ratio, finding higher ratios lead to increased returns. Anderson and Brooks [2006] studied long-term PE ratios, concluding multi-year earnings-based PE is a better predictor. There are studies that suggest that rapid PE growth compared to EPS in a rampant bull market might contribute to rise in stock prices. Overvaluation has led to stock bubbles. Notably, few studies have explored stock selection and prediction in this context. However, there is a demand for a holistic and a balanced methodology for stock selection for designing an optimal portfolio.

Our research employs a standard scorecard, as a benchmark for stock selection and subsequently uses the selection for portfolio construction and optimization.

3 Literature Survey

Survey of related works (Trend Prediction) Lee et al. [2009] employed support vector machine (SVM) and a hybrid feature selection technique for stock trend prediction. Using a subset of the NASDAQ Index in TEJD 2008, they adopted a hybrid approach involving sequential forward search (SSFS) for feature selection. Notably, the study offered a well-defined parameter adjustment procedure across various values. While the feature selection structure aided initial model structuring, a limitation was the exclusive comparison of SVM with back-propagation neural network (BPNN), omitting comparison with other machine learning algorithms. Ni et al. [2011] predicted stock trends using SVM and optimized through fractal feature selection. SSECI dataset with 19 technical indicators was employed, preceded by feature selection. Grid search with k cross-validation found best parameters. The study thoroughly evaluated feature selection methods. Their focus on technical indicators was notable. Kara et al. [2011] harnessed ANN and SVM for stock price index movement prediction using Istanbul Stock Exchange data (January 1997 to December 2007). A significant strength is their detailed parameter adjustment procedure, while their weaknesses include lacking novelty in indicators and model structure. Without clarifying performance superiority, more validation on other datasets is needed. They explained ANN and SVM functioning with stock features and recorded parameter adjustment.

Kim and Han [2000] merged artificial neural networks (ANN) and genetic algorithms (GAs) for stock price prediction, using technical indicators and daily KOSPI data (1989-1998). While optimizing ANN with GA, limitations included fixed input features and focused optimization. Qiu and Song [2016] also employed a hybrid GA-ANN approach for predicting Japanese stock market direction. Piramuthu [2004] rigorously evaluated diverse feature selection techniques across datasets like credit approval, loan defaults, web traffic, tam and kiang, assessing probabilistic and inter-class measures. Lei et al. [2018] used Wavelet Neural Network (WNN) with Rough Set (RS) for stock trend prediction, simplifying dimensions and shaping WNN structure using five indices. Huang and Tsai [2009] combined filter-based feature selection with self-organizing feature map (SOFM) support vector regression (SVR) for Taiwan index futures (FITX) trend prediction, enhancing efficiency through clustering training samples. While effective, training complexity posed challenges for shorter time intervals, and performance comparison was lacking.

Fischer and Krauss [2018] employed LSTM for SP 500 prediction, using month-end constituent lists from Dec 1989 to Sep 2015. They applied RMSprop as an optimizer, highlighting LSTM's superiority over DNN and logistic regression. However, their background lacked financial domain knowledge. The absence of LSTM training effort with long-time dependencies wasn't mentioned.

Tsai and Hsiao [2010] introduced a feature selection approach for stock prediction using TEJ database (2000-2007). Their method employed sliding windows, MLP-based neural networks, and feature selection techniques (PCA, GA, and CART), covering fundamental and macroeconomic indices. They evaluated methods based on combined model performance and statistical analysis.

Thakur and Kumar [2018] developed a hybrid trading support system using multi-category classifiers and random forest (RAF) on major indices (NASDAQ, DOW JONES, SP 500, NIFTY 50, NIFTY BANK). Their model combined random forest (RF) and weighted multicategory generalized eigenvalue support vector machine (WMGEPSVM) for "Buy/Hold/Sell" signals, incorporating RF-based feature pruning. The model provided investor signals but lacked direct stock expertise. Despite algorithm comparison, time and computational complexity were not discussed.

Idrees et al. [2019] proposed a time series-based approach for predicting stock market volatility. They emphasized feature engineering for ARIMA models through a three-step process: time series analysis, stationarity check, and parameter estimation using ACF and PACF charts. Nekoeiqachkanloo et al. [2019] introduced a comprehensive stock investment system with dual algorithms and time series-based forecasting. Their approach incorporates financial and technical features, but its evaluation on 25 prominent stocks might be limited due to shared features.

Survey of related works (Price Prediction) Karmiani et al. [2019] compared LSTM, Backpropagation, SVM, and Kalman filter for stock prediction from data of nine companies. LSTM showed higher accuracy and lower variance. Yu and Yan [2019] combined phase-space reconstruction with LSTM for market-specific stock prediction using historical data. Gao et al. [2020] favored Uncertainty-Aware At-

tention in a machine learning analysis of SP 500, CSI 300, and Nikkei 225 indices. Qiu et al. [2020] used LSTM with attention for news-based evaluation and wavelet denoising for index price prediction. Lanbouri and Achhab [2020] applied LSTM for high-frequency SP 500 price prediction. Yadav et al.[2020] enhanced Indian stock prediction with multi-layer LSTM, addressing trend and seasonality.

Chen et al.[2015] improved China stock return prediction by LSTM with 30-day sequences and ten features. Bao et al.[2017] denoised using Haar wavelets, employed stacked autoencoders, and used LSTM for SP 500 closing price prediction. Roondiwala et al.[2017] used LSTM for NIFTY 50, predicting closing prices based on fundamental data without macroeconomic or technical indicators.

Shen and Shafiq [2020] gathered two years of Chinese stock market data and introduced a thorough approach that merges feature engineering and deep learning for predicting price trends. Their solution covers dataset preparation, diverse feature engineering methods, and a custom deep learning model. Their model performance is attributed to the thorough feature engineering that was applied. A recent study by Bhandari et al.[2022] employs single layer LSTM to predict SP 500’s next-day closing price. Nine balanced predictors from fundamental, macroeconomic, and technical data were used in this study.

4 Volatility and Volatility Spread

Volatility, in a general sense, represents the evaluation of uncertainty levels. This uncertainty is closely tied to the volume of market information present. For instance, Mondays exhibit higher volatility compared to other weekdays due to the incorporation of weekend news (new information) into the first trading day. Several factors impact this uncertainty. Notable ones include rising interest rates, stricter policies by major central banks, inflation escalation, geopolitical conflicts, and disruptions in supply chains. This uncertainty in financial markets can be gauged pro-actively through implied volatility or retrospectively through realized volatility. In both cases, volatility is directly related to market information availability. Predicting volatility aims to anticipate information ahead of the market. Implied volatility predicts future price fluctuations in an asset based on market prices, particularly options. In practice, implied volatility (IV) is calculated from option prices of an underlying asset, indicating the market’s future volatility expectations for that asset. The VIX index is a popular choice for this. On the other hand, realised volatility measures actual price fluctuations in an asset over a specific past period using historical data.

Given the same information available, implied and realized volatilities with an identical time frame (e.g., one month) should theoretically be equal, representing the same volatility measure. However, due to the nature of their observation periods, implied volatility (reflecting future expectations) and realized volatility (observed after the fact) differ. This discrepancy stems from the fact that implied volatility anticipates future conditions while realized volatility is based on historical data. [Taylor S.J (2005) ; Rossi P.E (1996)]

This often leads to a contrast between implied volatility (market’s projection of future volatility) and later observed realized volatility, as they carry distinct information. Occasionally, the market accurately predicts volatility due to non-historical insights. Conversely, market volatility forecasts can be inaccurate, creating opportunities for volatility-based trades.

However, realized volatility is deterministic and thus stands as the most precise volatility measure. A logical estimation of future volatility emerges from the disparity between realized and implied volatilities.

A part of our work aims to forecast whether realized volatility will surpass implied volatility (positive RV-IV spread) in the subsequent trading day. Achieving this involves predicting the volatility spread between realized and implied volatilities. LSTM neural networks are employed to predict this spread’s direction, indicating if it will be positive or negative on the upcoming trading day. Specifically, a positive spread denotes an impending bear market due to expected heightened volatility.

5 Methodology

5.1 Dataset

The dataset is made of SP500 daily prices to calculate SP500 daily returns and realized volatility, and the VIX Index to get the SP500 implied volatility. Data is collected from the 2nd of January 2004 to the 29th of April 2020, for a total 4,109 daily prices. All the data is obtained from Yahoo Finance.

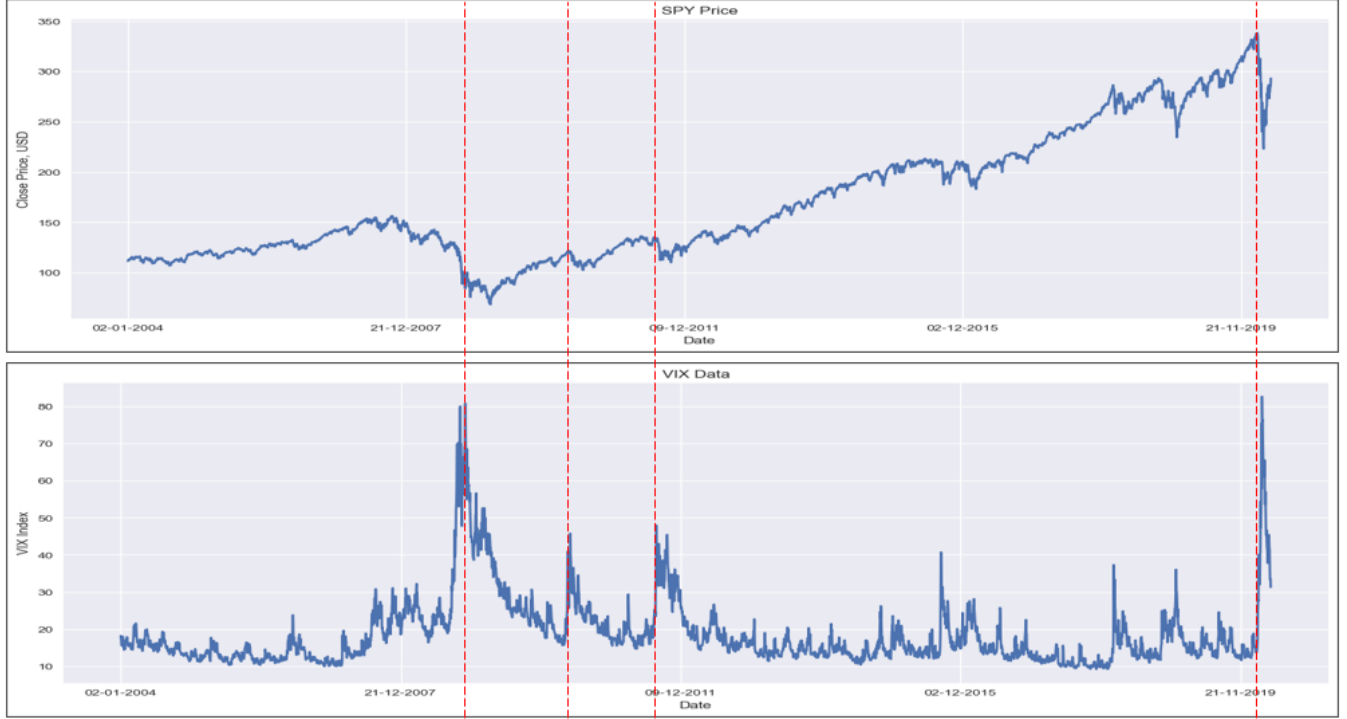


Figure 1: SP500 Prices and Volatility Index from 2004 to 2020 (*Source: Yahoo Finance*)

The red dotted lines are placed on the volatility 'peaks' during the period from 2004 to 2020 (the first one is the sub-prime mortgage crisis and the last one is the onset of COVID-19)

5.2 Target Variable

The label (target variable) is the SP500 RV-IV spread, which gets the value 1 if the realized volatility is higher than the implied volatility on the next trading day. Label '1' indicates a bear move (down trend) and label '0' indicates a bull move (up trend)

$$y_t = \begin{cases} 1 & : \text{if } \text{realised}_{t+1} > \text{implied}_{t+1} \equiv \text{RV-IV Spread} > 0 \\ 0 & : \text{otherwise} \end{cases}$$

where realised_{t+1} is the 1-day forward SP500 realized volatility, implied_{t+1} is the 1-day forward SP500 implied volatility, and $\text{RV} - \text{IV}_{t+1}$ the difference between the two. Over the 4,109 days of available volatility figures, the realized volatility was above the implied volatility on a total of 533 days, so around 13 % of the time. At this stage, the dataset is imbalanced. Figure 2 shows the volatility spread over the mentioned period of study.

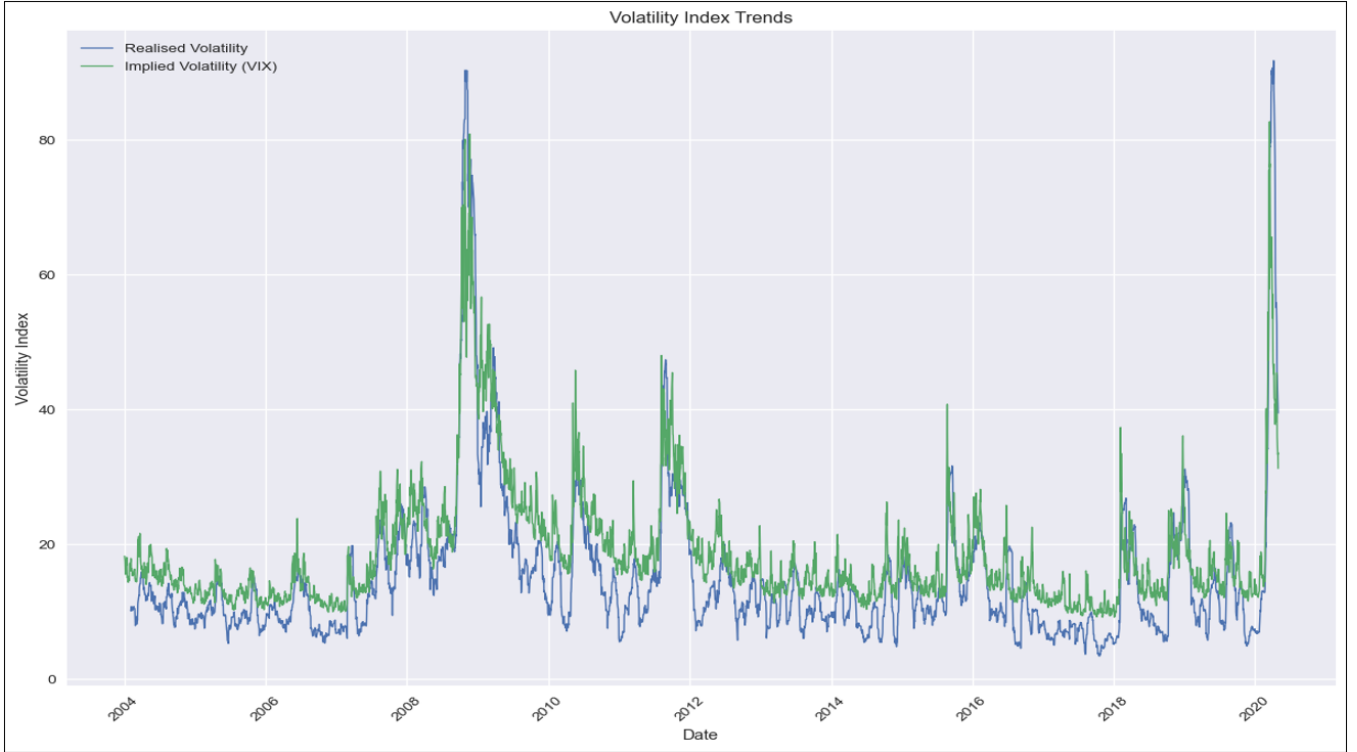


Figure 2: Volatility Spread from 2004 to 2020

5.3 Feature Engineering

Now our base dataset contains SP500 close prices, VIX, computed realised volatility and the target variable signal (0,1). A feature expansion exercise was performed on this base dataset using pandas ta. After performing some basic clean-up, there were finally 218 features available in the expanded dataset. Post this, a 'class balancing' exercise was performed on the dataset and then all time series were split into a train set of 80 % for model training and test set of 20 % for model validation purposes.

Boruta feature selection technique [Witold et al.(2010)], employs a random forest classifier to assess feature importance. It compares each feature's significance with a set of randomly generated shadow features. The process iterates, retaining features with higher importance scores than shadows and discarding others. The final feature set is composed of those consistently outperforming shadows across iterations.

170 features out of the 218, were classified as high weighted and relevant in our final feature set, for RV-IV classification result. Since, almost 80 % of the features were selected, we attempted performing dimensionality reduction, using self-organising maps.

5.4 Dimensionality Reduction (SOM)

A self-organizing map serves as an unsupervised learning technique. It involves categorizing sample data points by their feature vectors' similarity. These points are then often plotted onto a 2D grid to visually identify shared characteristics. [Wilmott.P (2020)]

We performed a dimensionality reduction technique on our feature set, using SOM. The results are as shown in figure 3. Darker regions in the heatmap represent larger distances between neighbouring neurons, indicating potential cluster boundaries. Lighter regions indicate smaller distances, suggesting smooth transitions between clusters.

In the feature set, there were no features with 'high correlation'. Prior to performing SOM, the feature correlations were checked (keeping the threshold as 0.75). Since, all the features were reasonably optimally correlated, the results from the dimensionality reduction exercise were not conclusive.

Hence, for the LSTM model building exercise, the entire expanded feature set of 218 features, was chosen for the training purposes

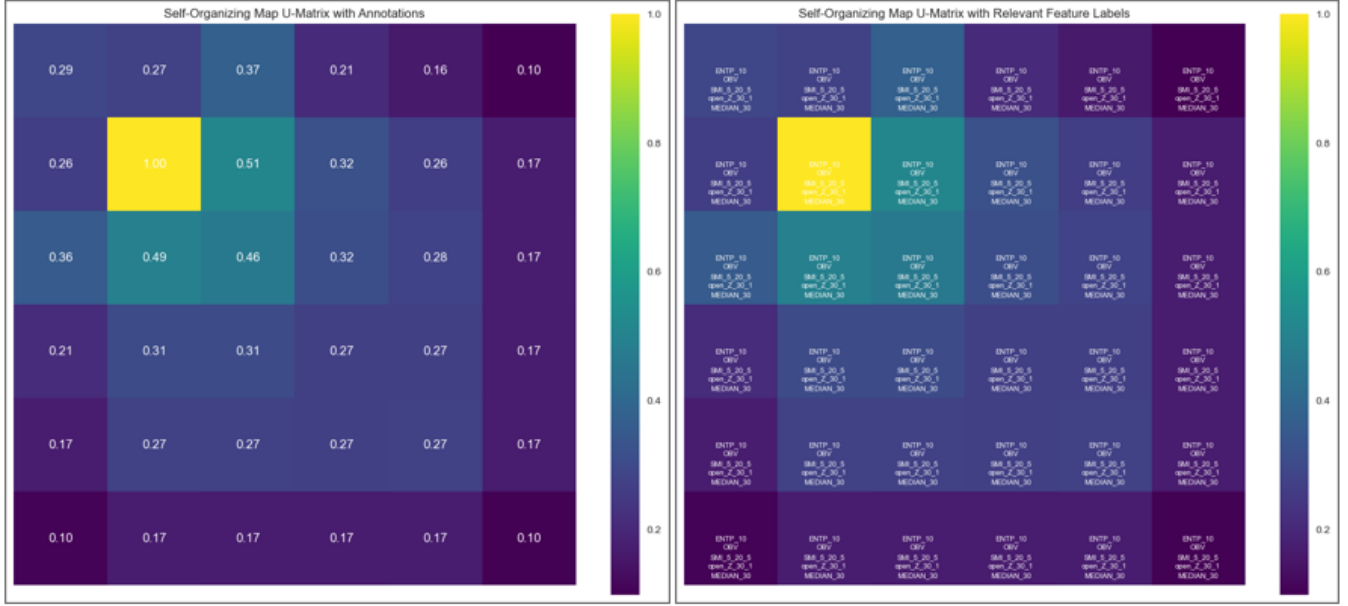


Figure 3: Self-Organizing Map U-Matrix

5.5 Architecture

The primary architecture of our proposed solution can be partitioned into three key components. Initially, the focus is directed towards the selection of effective features that assume a crucial role. Next, the data is closely scrutinized, and dimensionality reduction is executed to streamline complexity. The culminating and most significant aspect of the work entails the construction of a model specially tailored for the analysis and forecasting of target stock behavior based on the target variable. This comprehensive approach finds its visual representation in Figure 4, providing a high-level depiction of the structure of our innovative solution.

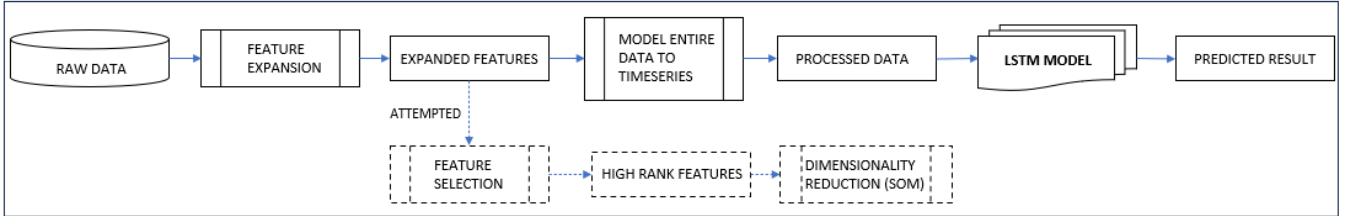


Figure 4: Architecture Flow Diagram

5.6 Algorithm

This section describes and elaborates the primary algorithms that are used in this work. Figure 5 provides the summary of the algorithms.

5.6.1 Algorithm Elaboration : Algorithm 1 (LSTM Model Building)

Elaboration of Algorithm 1 (LSTM Model Building)

The process begins by loading the data and defining the target variable to predict. Additional features are generated using the Pandas TA library to enhance the dataset. Addressing class imbalances, proper weights are assigned to each class. Subsequently, the data is divided into training and testing sets, maintaining an 80:20 split ratio. For training and testing, sequence data is prepared using the Timeseries Generator, tailored for time series analysis.

Transitioning to model creation, a step-by-step construction of a sequential model occurs. Convolutional Neural Network (CNN) layers, which identify data patterns, can be optionally added. To capture

sequential patterns, Long Short-Term Memory (LSTM) layers are integrated. Model performance is optimized through a designated optimizer, and the model is compiled, incorporating a chosen loss function and evaluation metrics.

Training is executed on scaled training data for a specified number of epochs, with the observation of training loss and the consideration of class weights. The strategy involves iterative model training, loss monitoring, and evaluation against test data, iterating until a stopping criterion is met or the maximum number of epochs is reached (patience of 20 iterations).

Conclusively, the trained model undergoes evaluation, involving a classification report and metrics like accuracy, precision, and recall. This comprehensive procedure encompasses data preprocessing, model building, iterative training, and evaluation, culminating in a deep comprehension of the model's predictive capabilities.

STEP 1: Load Data STEP 2: Define target variable STEP 3: Generate Features (pandas ta) STEP 4: Correct class imbalances using class weight function Weight A = $(1/\text{class A}) * (\text{length of data frame}) / 2$ STEP 5 : Test – Train split (80:20) on entire dataset STEP 6: Generate train and test sequence data (TimeseriesGenerator(scaledtrain, ytrain, length)) STEP 7: Create a sequential model <pre>def create_model (nuerons): STEP 7.1: Instantiate the model model = Sequential() Add CNN Layers model.add(CNN LAYER (filters, kernel size, activation, input shape) model.add(CNN Pooling (pool size) STEP 7.2: Add LSTM Layers model.add(LAYER1 (units, activation, dropout) model.add(LAYER2 (units, activation, dropout) model.add(LAYER3 (units, activation, dropout) STEP 7.3: Specify optimizer separately (preferred method) opt = Adam(learning_rate, epsilon) STEP 7.4: Model compilation model.compile(optimizer=opt, loss , metrics=['accuracy', Precision(), Recall()]) STEP 7.5: Model fitting model.fit (scaledtraindata, epochs=500, callbacks, class weight) For "choice of layers and neurons", Do For "range of number of replicates", Do Train the model, monitor training loss; Continue Until training loss at epoch $n \leq$ training loss at epoch $n + 1 \leq \dots \leq$ training loss at epoch $n + 19$, Or maximum epochs are reached. Evaluate model on the test data. Calculate metrics. End Do. STEP 8: Evaluate Model (Classification report, Metrics)</pre>	1] Instantiate the model <pre>model = Sequential()</pre> 2] Tune the number of units in the layers <pre>hp_units = hp.Int (min_value, max_value, step)</pre> 3] Tune the dropout rate <pre>hp_dropout1 = hp.Float ('Dropout_rate', min_value, max_value, step)</pre> 4] Tune activation functions <pre>hp_activation1 = hp.Choice (name = 'activation', values = ['relu', 'elu'], ordered = False)</pre> 5] Tune CNN features <pre>hp_filters, hp_kernel_size, hp_pool_size = Int/float (min_value, max_value, step)</pre> 5.1] Add CNN Layers <pre>model.add (CNN LAYER (filters, kernel size, activation, input shape) model.add (CNN Pooling (pool size)</pre> 6] Add LSTM Layers <pre>model.add (LAYERS.. (units, activation, dropout)</pre> 7] Choosing the optimizer <pre>opt = hp.Choice('optimizer', values=['adam', 'adadelta', 'nadam']) if opt == '<chosen optimizer>': # Define hyperparameters for <chosen optimizer> (learning rate, epsilon)</pre> 8] Chosen optimizer with hyperparameter values <pre>opt = <chosen optimizer> (learning_rate, epsilon)</pre> 9] Model compilation <pre>For "choice of optimizers", Do For "choice of chosen parameters", Do For "range of number of replicates", Do Train the model, monitor validation loss; Continue Until training loss at epoch $n \leq$ training loss at epoch $n + 1 \leq \dots \leq$ training loss at epoch $n + 5$, Or maximum epochs are reached. Evaluate model on the validation data Calculate Metrics. End Do. Output Set of best hyperparameters, with metrics.</pre>
---	---

Figure 5: Algorithms used in this work

5.6.2 Algorithm Elaboration : Algorithm 2 (Hyperparameter Tuning)

Elaboration of Algorithm 2 (Hyperparameter Tuning)

This algorithm guides the creation and enhancement of a neural network model using hyperparameter adjustments. It starts by forming the model's structure, fine-tuning elements like units and dropout rates. Activation functions are varied, and attributes of CNN and LSTM layers are explored. Moving forward, the focus turns to configuring the model for optimization. Optimizers are chosen and tailored, involving options like 'adam' or 'adadelta'. Parameters like learning rate are adjusted for optimization. Model training and assessment take place subsequently. Different optimizer choices are tested, each with various

parameter sets and replicates. Training is iteratively conducted, and validation loss is monitored to ensure optimal convergence (patience of 5 iterations). Model performance is evaluated using validation data, and various performance metrics are computed.

As a result, the algorithm yields a set of optimal hyperparameters alongside their associated metrics. In summary, this process systematically shapes the neural network model by tweaking hyperparameters to attain the highest performance potential.

6 Experiments and Results

6.1 Base Classifier build

After the feature selection process, a classification exercise was performed using random forest algorithm, as a precursor to LSTM model building. This exercise was performed in two phases, phase 1 with the entire expanded feature set (with 218 features) and phase 2 with selected feature set from Boruta (170 features). The target variable definition was kept the same as mentioned earlier. For phase 1, the predictor signal (0,1) from the target variable was vectorised as 'y' and the original expanded feature set (218 features) was taken as the training matrix 'X'. The train : test split was maintained as 80:20. The class weights were applied to address class imbalances and finally the test datasets of [X : y] were fit using a random forest classifier. The results of the classification are shown in figure 6 (left pane).

For phase 2, the filtered feature set out of Boruta (with 170 features) was taken as the training matrix 'X' instead of the original feature set. Everything else were kept the same, as of phase 1. The results of the classification are shown in figure 6 (right pane).

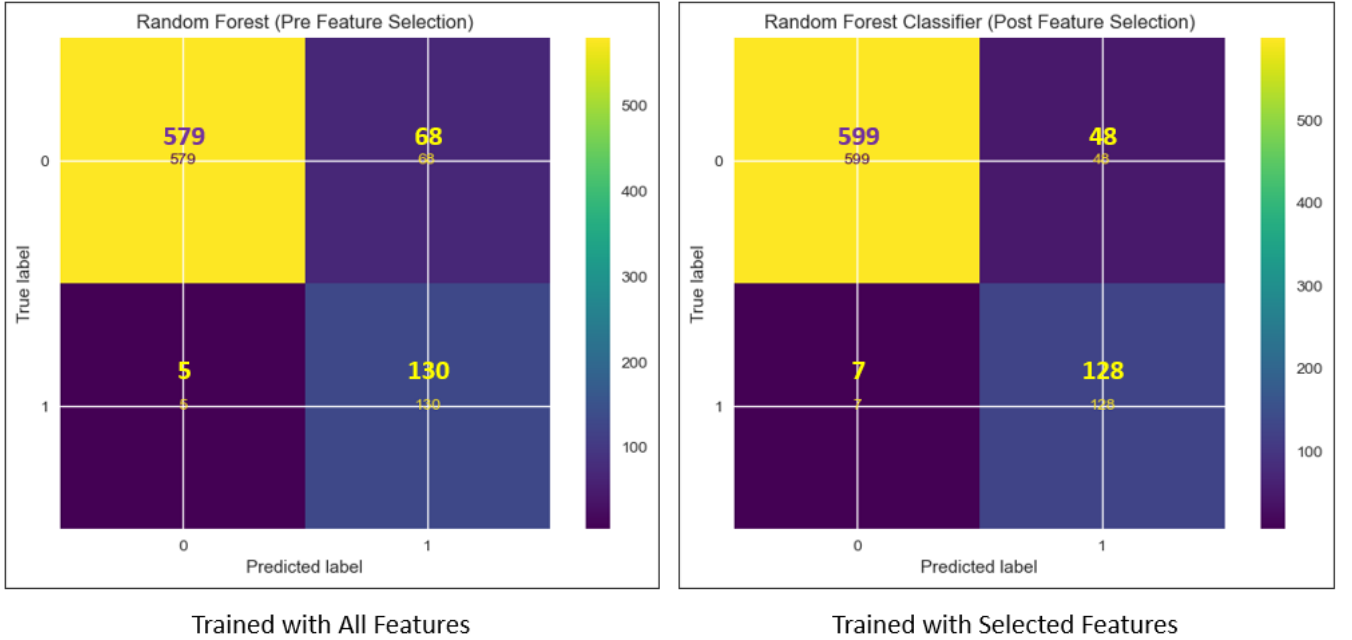


Figure 6: Base Classifier (Pre vs Post Feature Selection)

6.2 LSTM Model building

Prior to the actual LSTM model building exercise, the entire expanded dataset was chosen as the as the training matrix 'X'. The predictor signal (0,1) from the target variable was vectorised as an array, 'y' and the train : test split was maintained as 80:20. Class weight functions were defined as class imbalances were addressed appropriately. Post this, for training and testing purposes, sequence data was prepared using time series generator. The original dataset matrices were scaled using min-max scaler, to suit time series generation.

For LSTM model building, the steps were carried out exactly as described in algorithm 1. The parameters, that remained fixed during the entire LSTM model building, are as given in Table 1.

Model Parameters	Value
Neurons	10
Lookback	60
Activation (output layer)	sigmoid
Epochs	500
Patience(For early stop)	20
No. of layers	3
Environment and Configuration	Value
RAM and Storage	16 GB and 1 TB
Environment	Python 3.9.13 ; TensorFlow ; Keras APIs
Operating System	Windows 10 Home (64 Bit OS)
Processor	Intel i7 Core (CPU:1.80GHz)

Table 1: Fixed Parameters and System Settings

The no. of units, that were chosen for the LSTM layers were of the order of 256 and 512 (layer 1). The optimiser that was chosen was adam with learning rate of 0.001 and epsilon of 1E-08.

A small array operation was performed to normalise the 'test' and 'predict' vectors, in order to generate the classification report and the confusion matrix. The final data points, on which the 'classification report' is presented is 722. However, the original test vector has 786 data points. This exercise is performed to correct the minor issues in a 'classification -prediction problem'. To accommodate for this, considering a 1 % tolerance, on the results that we may see in the classification report for the LSTM models, is recommended.

The defined metrics for model performance were accuracy, precision and recall (as default values given in the algorithm). Binary cross entropy technique was chosen as the criteria for loss detection.

6.2.1 Hyperparameter Tuning

Post the model fitting exercise with the pre-tuned model, an intensive hyperparameter tuning exercise was performed on all other possible parameters (apart from the parameters in table 1), on the LSTM model.

All possible paramaters for creating the model sequence like no. of units, dropout rate, activation methods , were chosen for tuning. Also three best suited optimizers (adam, adadelata, adadelata) for this specific problem, were chosen as a part of tuning and all possible parameters, relevant to each optimizer, were also chosen for tuning. A detailed study of the [keras.io documentation] was performed, in order to devise the hyperparameter tuning strategy. A dynamic and a flexible function was written to build the tuned model. Three popular hyperparameter tuning methods viz. random search, hyperband and bayesian optimisation, were chosen. The LSTM model was tuned on each of these methods individually multiple times. The results from tuning the LSTM model, are given in table 2.

6.2.2 CNN + LSTM Model building

Following the model building and tuning exercise, as mentioned in the previous sections, a similar exercise was performed with a hybrid model containing a layer of CNN above the LSTM layer. Hence, in this part of the experiment, the additional parameters relevant to the CNN layer, were also included in the hyperparemeter tuning exercise. The model building and tuning exercise were performed using a similar dynamic tuning function, written for this exercise. The CNN + LSTM model was tuned on all parameters with the same three chosen tuning methods multiple times. The results from the hyperparemeter tuning exercise are shown in table 3.

A special mention : In table 2, there is a column named HYP100. This is a special intensive tuning exercise, that was performed on the LSTM model using the hyperband method. The speciality of this method is that it was tuned on 50 epochs (instead of a default 3-5 epochs), on 100 trials using the hyperband algorithm. This was performed to ensure that the hyperparameter tuning exercise is as exhaustive as

Hyper Parameters	Random Search	HyperBand	Bayesian	HYP100
Units (Layer 1)	12	24	4	32
Units (Layer 2)	8	28	24	4
Units (Layer 3)	20	28	24	24
Dropout Rate	0.3	0.3	0.2	0.0
Activation	elu	elu	relu	elu
Optimizer	adam	adam	adadelta	nadam
Learning Rate	0.0001	0.0001	0.001	0.01
Epsilon	3.71E-8	2.71E-6	1.43E-7	1.03E-8
Rho	NA	NA	0.88	NA
Beta 1	0.84	0.98	NA	0.97
Beta 2	0.81	0.91	NA	0.88

Table 2: Results from Hyperparameter Tuning (LSTM Model)

Hyper Parameters	Random Search	HyperBand	Bayesian
Units (Layer 1)	8	32	28
Units (Layer 2)	20	4	16
Units (Layer 3)	28	8	20
Dropout Rate	0.0	0.4	0.1
Activation	relu	elu	relu
Optimizer	adam	adam	adam
Learning Rate	0.01	0.001	0.01
Epsilon	1.77E-7	1.84E-8	3.16E-7
Filters	12	20	36
Kernel Size	2	2	2
Pool Size	2	2	6

Table 3: Results from Hyperparameter Tuning (CNN+ LSTM)

possible. The complete list of performance metrics from the model building exercise, is given in table 4 and visually presented in figure 7.

Model	Accuracy	Precision	Recall	F1 Score
Base (Pre-Selection)	0.91	0.82	0.93	0.86
Base (Post-Selection)	0.93	0.86	0.94	0.89
LSTM (Pre Tuned)	0.90	0.72	0.72	0.72
LSTM (Random Search)	0.86	0.76	0.71	0.74
LSTM (HyperBand)	0.90	0.72	0.63	0.67
LSTM (Bayesian)	0.81	0.73	0.86	0.79
CNN+LSTM (Pre Tuned)	0.90	0.71	0.64	0.67
CNN+LSTM (Random Search)	0.77	0.73	0.56	0.62
CNN+LSTM (HyperBand)	0.83	0.71	0.56	0.62
CNN+LSTM (Bayesian)	0.81	0.74	0.86	0.79
LSTM (HYP100)	0.81	0.74	0.86	0.79
Single Layer	0.99 (R-Squared)			

Table 4: Performance Metrics

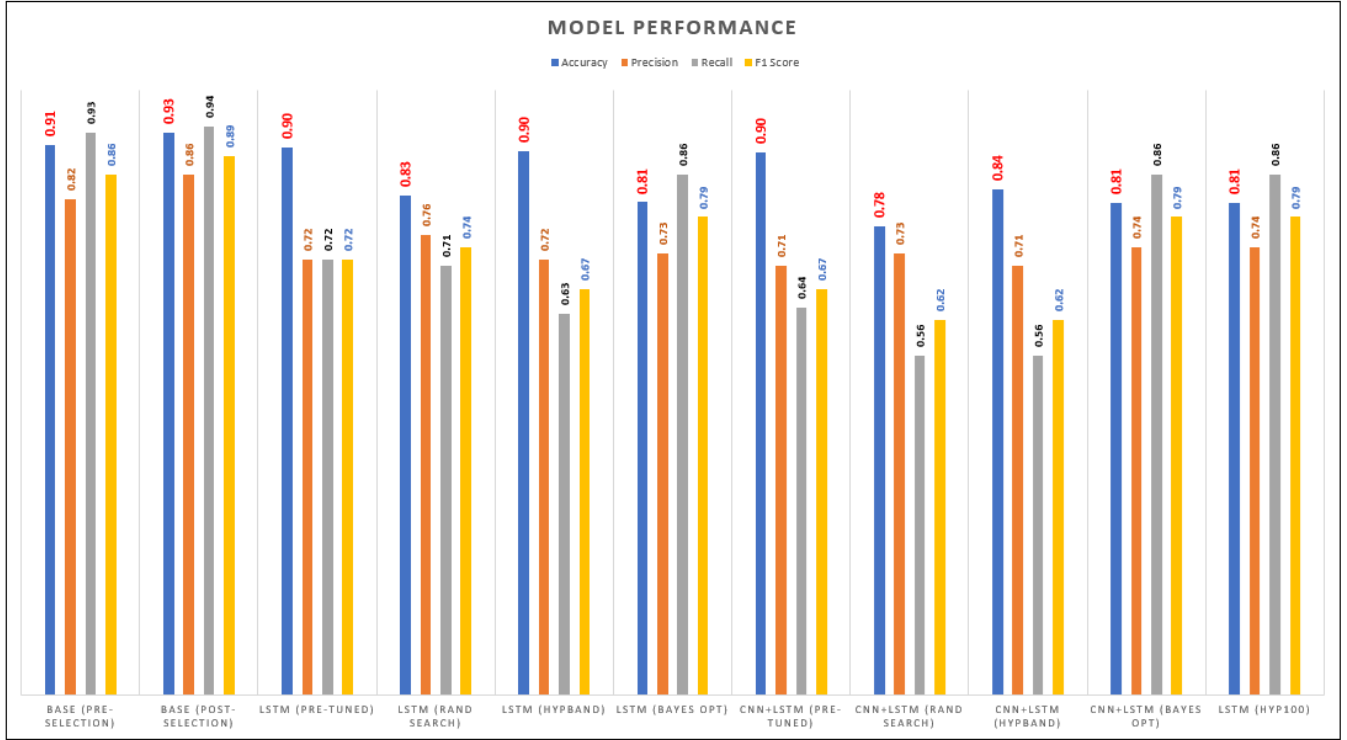


Figure 7: Model Performance

6.3 Single Layer LSTM for Prediction

Post an extensive model building exercise that was performed using multilayer LSTM, an alternative approach was considered to attack the problem statement from a 'prediction accuracy' perspective.

Hyper Parameters	Single Layer	Multi Layer (HyperBand)
Activation	relu	elu
Optimizer	nadam	adam
Learning Rate	0.001	0.0001
Epsilon	1.01E-8	2.71E-6
Beta 1	0.99	0.98
Beta 2	0.90	0.91
Accuracy	0.86 (R Squared)	0.89

Table 5: Key set of Optimised Parameters

An experiment was performed to predict the price and volatility directly (instead of trend) by means of LSTM. A single layer LSTM model was chosen for this [in reference to Bhandari et al. (2022)] and the tuning was performed using random search algorithm only. The results of the model performance (post tuning) were impressive with a consistent R-squared score of 0.99 on price and 0.86 on volatility index (VIX).

The results of the model performance are shown in figure 8. Finally, the key set of optimised paramaters is presented in table 5.

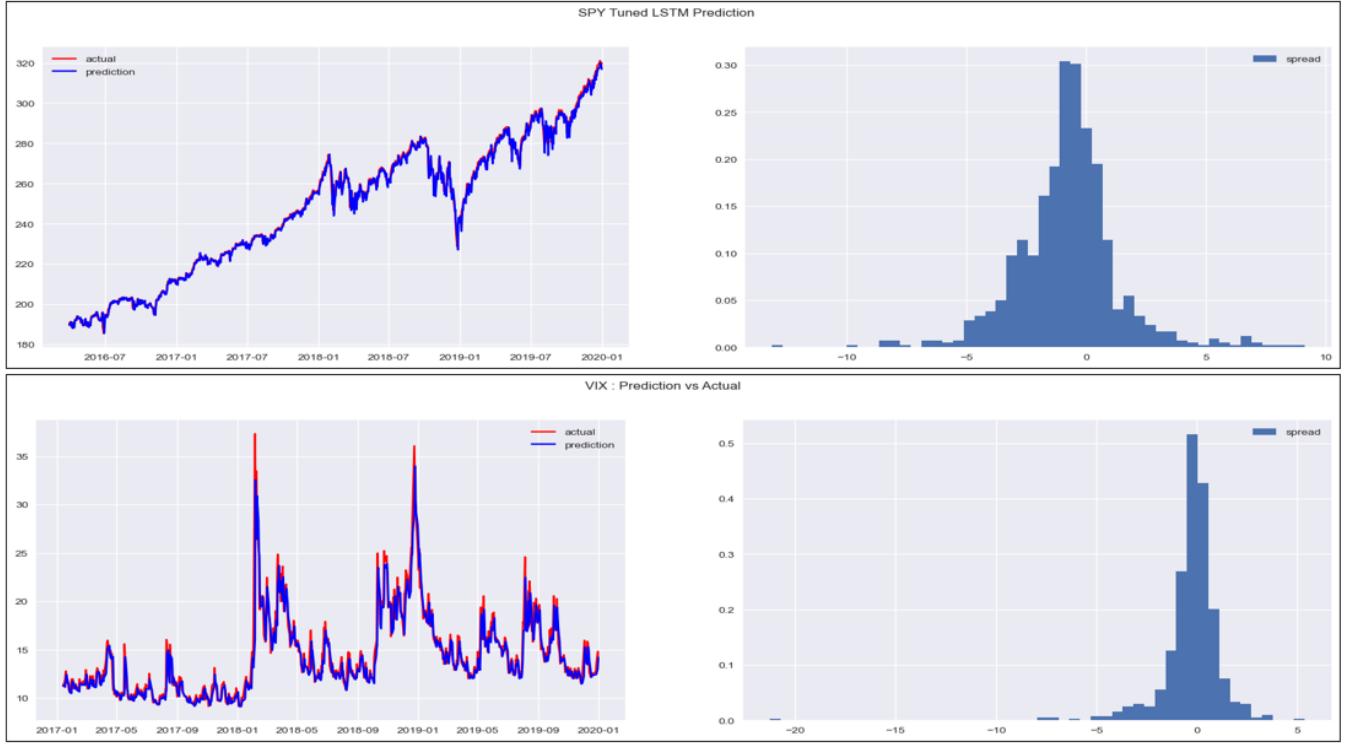


Figure 8: Prediction (Price vs Volatility)

7 Discussion

7.1 Base Classifier

At the outset, we see that the performance of base classifier is really good, when it comes to our classification problems. Referring to line items 1 and 2 in table 4, we see that the classification accuracy is very good, especially on the filtered dataset (with 170 features). It is also worth verifying this from figure 6. We can clearly see from figure 6, that the classification is pretty good for our base classifier (random forest).

Since we are aiming to predict whether a price movement will be positive (up trend) or not, it can be considered a classification problem. Classification involves predicting the category or class to which an observation belongs, based on the input features. In our case, even though the output is a prediction of the future price movement, the task is to assign a categorical label (positive or negative) based on the given features. [Lee et al. (2009)] and [Ni et al. (2011)] has used support vector machine (SVM) to predict stock trends and have published promising results.

However, usage of random forest algorithm to predict stock trends was not very evident in recent literature and hence we have chosen it. The results from our work w.r.t the base classifier is comparable the work of Ni et al. and Lee et al. The promising performance of random forest classifier could be attributed the ability of the classifier to capture patterns and feature interaction, in relatively simpler, non-linear datasets (like in our case) and their robustness in handling diverse datasets [Breiman.L (2001)].

7.2 LSTM and CNN+LSTM Models

7.2.1 Pre-Tuned (LSTM vs CNN+LSTM)

We first will compare the results of the pre-tuned models of LSTM vs CNN+LSTM. In terms of accuracy, both of them are giving close to 90 %. Looking closely into the details of classification (figure 9), the results , we observe that the performance of the LSTM model is better than the CNN+LSTM model. The effort of adding an extra CNN layer seems to limit the model performance. A possible reason could be that complex models can degrade when faced with small datasets [Shwartz and David (2014)]

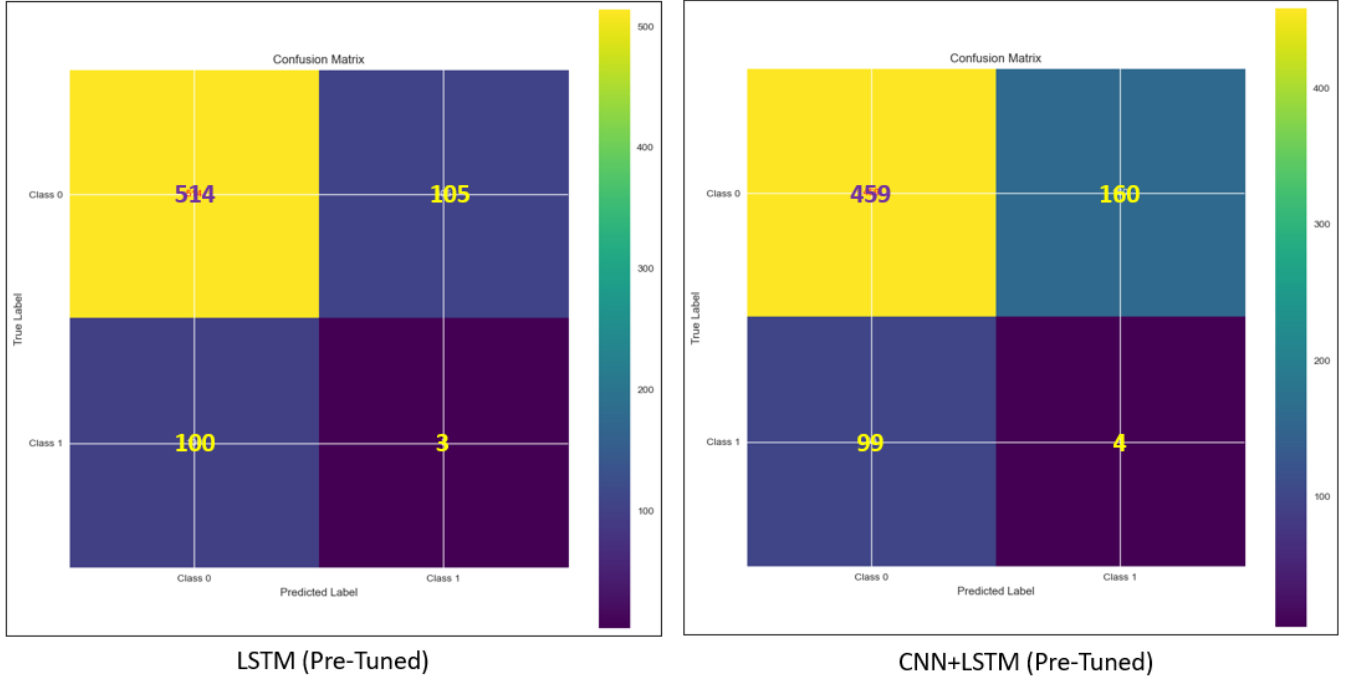


Figure 9: Pre Tuned (LSTM vs CNN+LSTM)

7.2.2 Tuned Models (Random Search vs Hyperband vs Bayesian)

As we proceed to the to view the confusion matrices of the two models across figures 10 and 11, we see a similar kind of a result. However, it is clear that post tuning, the accuracies of the CNN+LSTM models are dropping as compared to the pre-tuned models. And across the two combinations, the hyperband algorithm seems to perform better than random search algorithm. The chosen optimizers for both were 'adam' and all the other parameters are almost similar (refer table 2). The only possible difference was that the no. of units in the different LSTM layers were different. A similar observation is evident for the CNN+LSTM model (from table 3). A possible reason for the slightly better performance of hyperband could be attributed to the algorithm's ability to allocate resources effectively, balance exploration and exploitation, and adapt to the performance of previous configurations. [Li and Jamieson et al. (2018)] Here too, we observe that the classification performance dips for the CNN+LSTM model, as compared to the LSTM model.

We now proceed to figure 12, to observe the classification performance of the bayesian optimisation method. Here, we observe that the model seems to overfit clearly across both flavours (LSTM and CNN+LSTM). A possible reason (considering the fact that our dataset is not very complex) could be that bayesian optimization uses a surrogate model to approximate the objective function. If the dataset is not very complex, a complex surrogate model (e.g., Gaussian process) might lead to overfitting due to the limited data [Snoek et al. (2012)]. On the contrary, hyperband's strength lies in efficient resource allocation, which can work well even with simpler models. It might be more robust in cases where the objective function is not highly complex. [Li and Jamieson et al. (2018)]

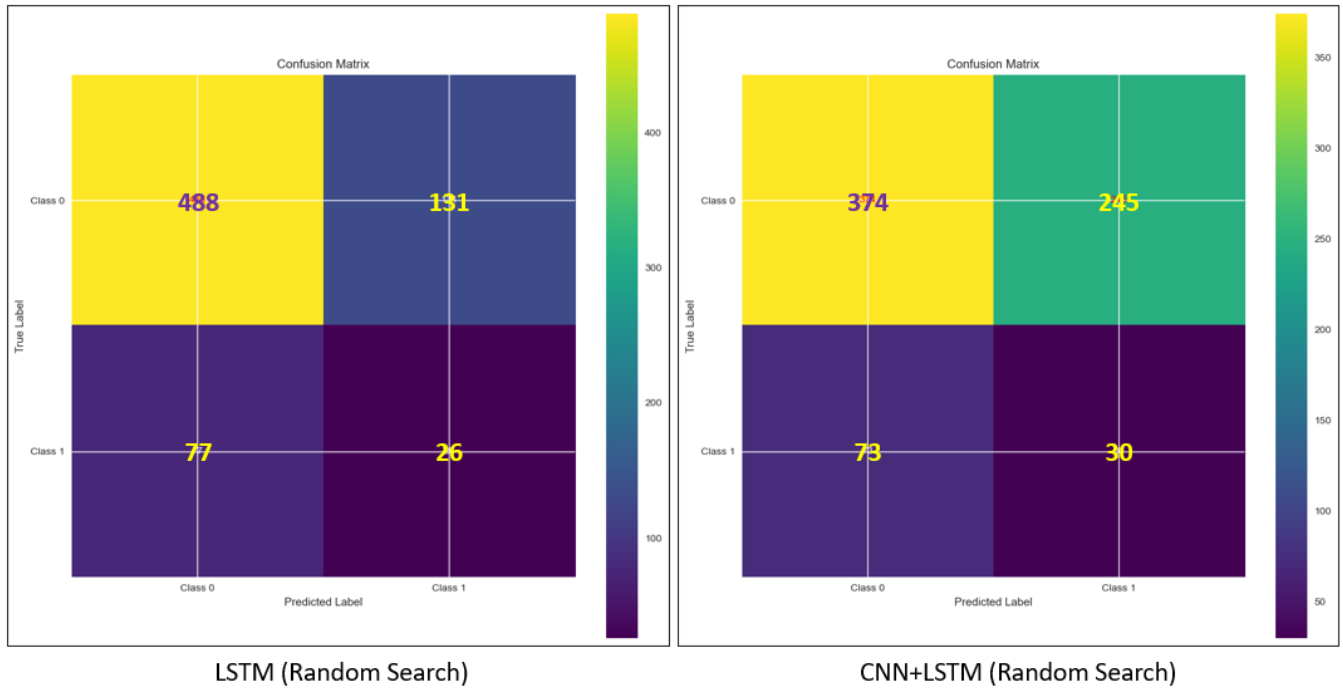


Figure 10: Random Search Tuned (LSTM vs CNN+LSTM)

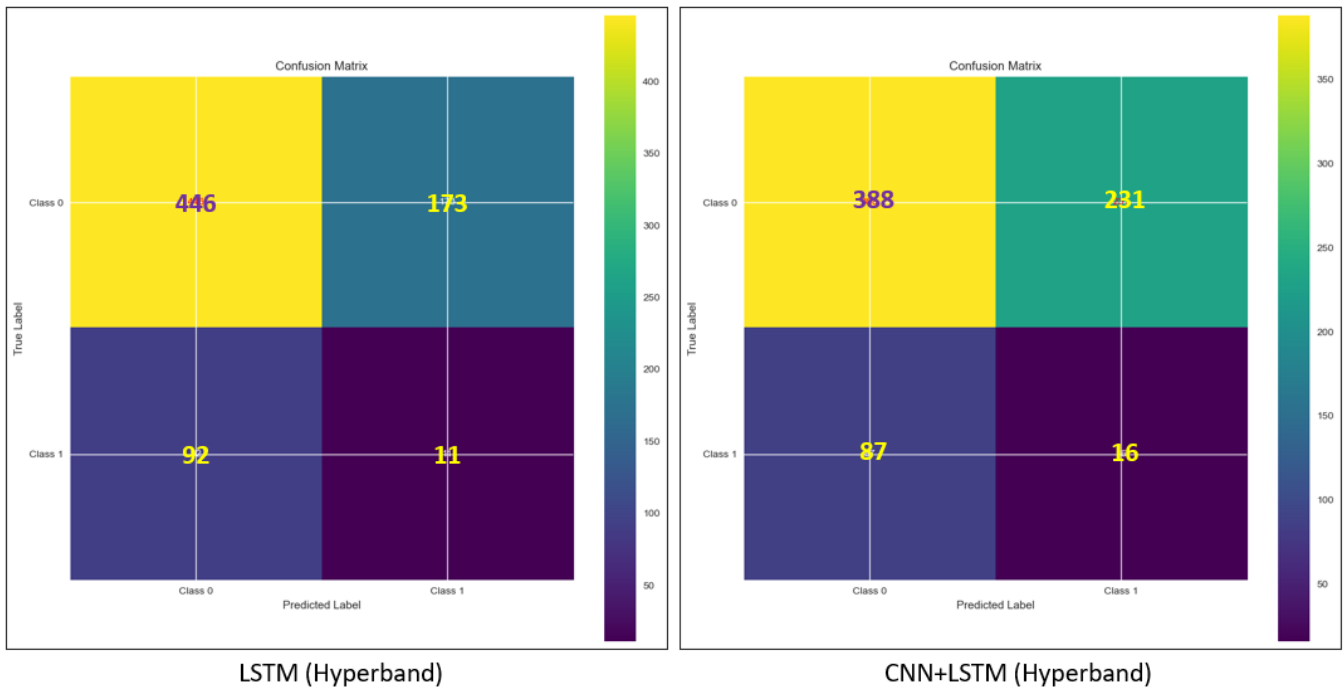


Figure 11: Hyperband Tuned (LSTM vs CNN+LSTM)

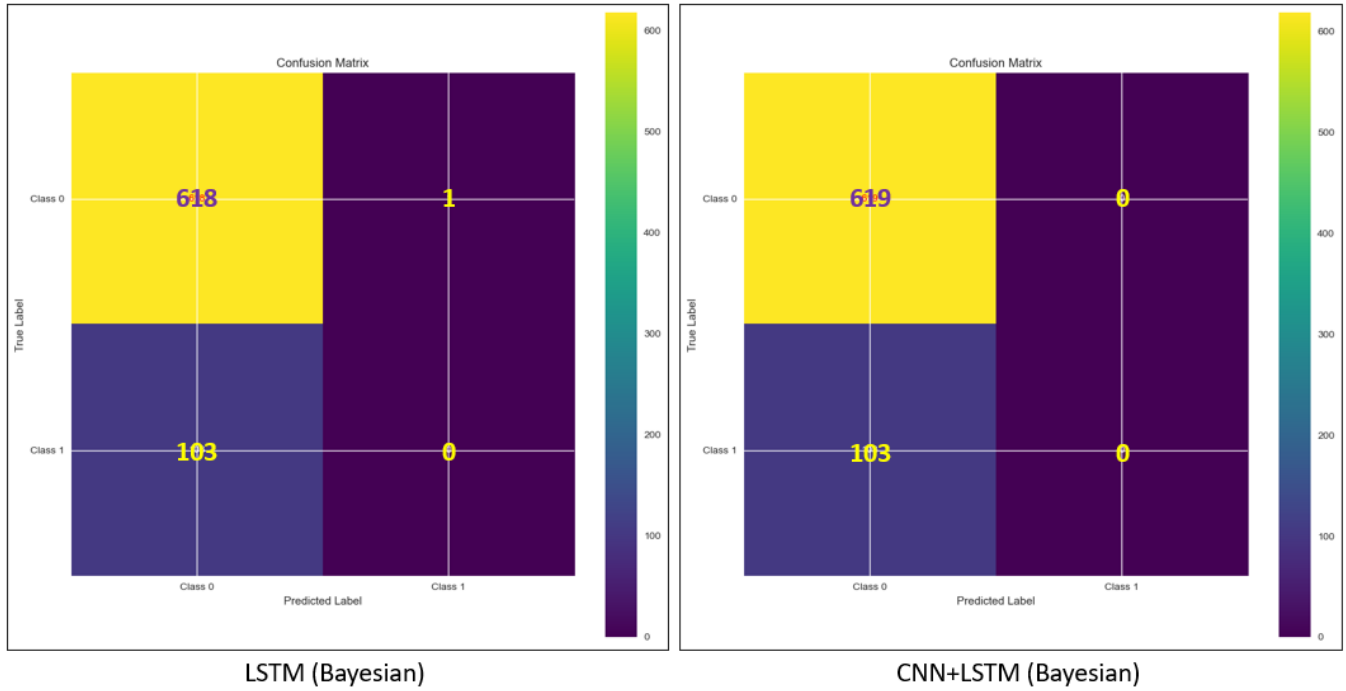


Figure 12: Bayesian Tuned (LSTM vs CNN+LSTM)

Hence, to make our analysis as exhaustive as possible, we performed our final tuning based on parameters from 'HYP100' combination. (Refer to table 2). Since, the CNN+LSTM combinations were consistently low performing, we performed the HYP100 tuning, only on the LSTM model. The no. of epochs was set at 50 (default being 3-5) and the no of trials was set for 100. The final comparison between our base classifier and LSTM (HYP100), is given in figure 13. This now re-substantiates that the now tuned LSTM model seems to overfit the data as compared to the base classifier (which performs a balanced and accurate classification).

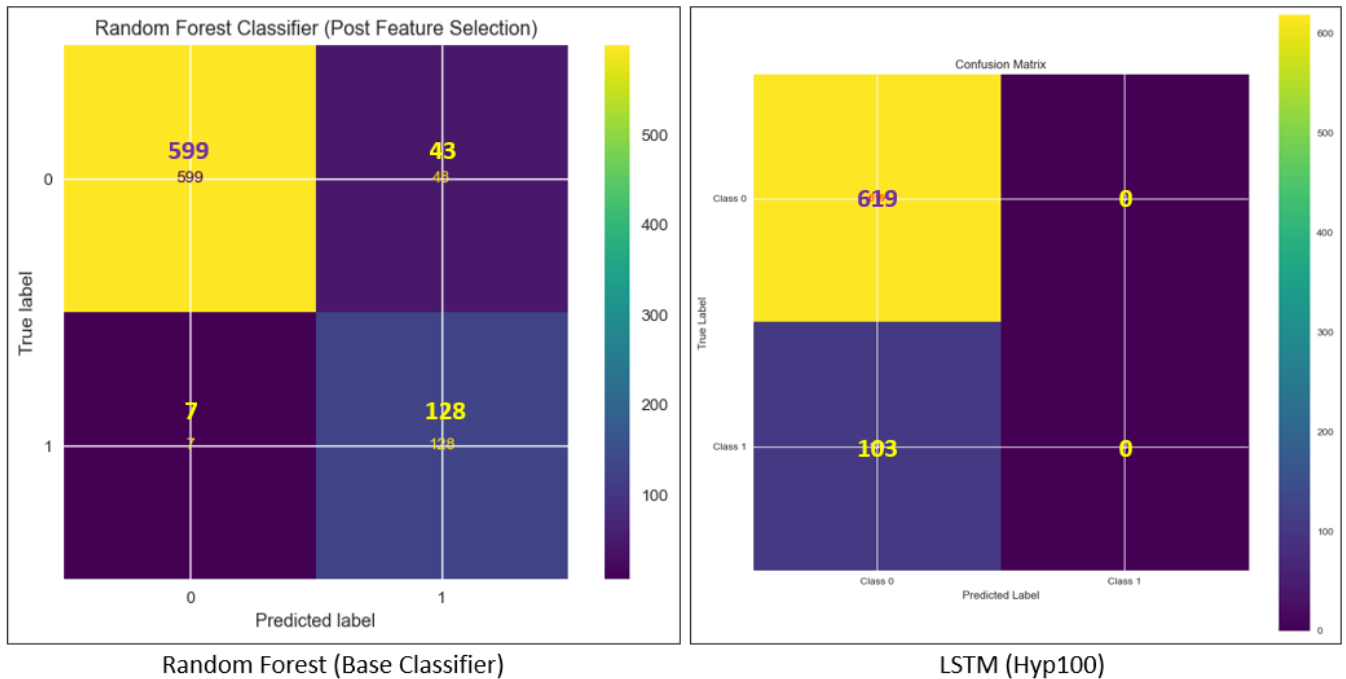


Figure 13: Base Classifier vs LSTM

LSTM vs Base Classifier A key observation across all LSTM based classifications is that, the classification seems to be imbalanced. If we closely observe figures 9-13, the '0' labels are accurately classified as against the '1'. For the base classifier, the classification of true vs predicted is far more balanced (very low false positives / negatives) as compared to the LSTM models. There could be multiple reasons that can be attributed towards this.

1] LSTM models are particularly well-suited for capturing temporal patterns and dependencies in sequential data while random forest models are well suited for capturing important patterns in the simpler datasets. [Hochreiter and Schmidhuber (1997)]

2] Simpler algorithms can sometimes outperform more complex models in certain scenarios (like in our case, wherein the dataset is simpler). [Shwartz and David (2014)]

3] Bias-variance trade-off and overly complex models can lead to overfitting, while simpler models might generalize better.[Bishop C.M (2006)]

[Shen and Shafiq (2020)] have performed an extensive work on LSTM for stock trend prediction and have shown promising results. Our LSTM framework is having an average accuracy of 0.85 - 0.90 as against [Shen and Shafiq (2020)] where they are averaging 0.93. Also their other classification results are more balanced. A plausible reason for this could be that the dataset that was used in their study was much more comprehensive wherein they have integrated trading data, financials, shareholders information etc., into one single dataset. Also an extensive feature engineering exercise was performed by [Shen and Shafiq (2020)] prior to model building exercise. This could have possibly helped their LSTM model to perform better since the large dataset offers a better space for the LSTM to form spatio-temporal connections between different features [Hochreiter and Schmidhuber (1997)]. However, [Shen and Shafiq (2020)], have not focussed much on the aspects of hyperparameter tuning.

Since, the focus of our work is largely on model building and considering the cost of constructing, maintaining large datasets, the practical constraints in data availability, we have considered a reasonably simpler dataset, with just the SP500 price and VIX.

7.3 Single Layer LSTM for prediction

[Hochreiter and Schmidhuber (1997)] and [Bhandari et al. (2022)] are now making it clear that LSTM models are well suited for prediction problems. Moreover, [Bhandari et al. (2022)] are recommending that a simpler model like a single layer LSTM is best suited for datasets like in our case, for attaining prediction accuracy. The parameters that were used are given in table 5 and results of prediction are evident in figure 8. The key highlight of is that our single layer LSTM model is performing well not only for price predictions but also for volatility predictions. In addition to the possible reasons mentioned earlier, the strengths of LSTM models in capturing sequential and temporal dependencies in data could be one allied reason here. [Olah C. (2015)]

Although, [Bhandari et al. (2022)] have performed an extensive analysis, the prediction was only on SP500 the close price and not on VIX. However, the ability to predict volatility could be key to determine market trends [Taylor S.J (2005)]. Hence, we chose to perform predictions both on price and volatility (VIX). It is key to note here that the model consistently was able to provide an accuracy of 0.99 (for price prediction) and 0.86 (for volatility prediction), refer figure 8. Moreover, [Bhandari et al. (2022)] did not focus much on feature engineering, although their dataset was slightly more complex than ours. However, their problem statement was purely 'prediction accuracy' based.

Whereas we intended to performed a detailed analysis on what would suit better for classification vs prediction based problems, for simpler datasets.

8 Volatility vs Market Sentiments

Although our base dataset is non-complex, when dealing with volatility based predictions (as in our case) considering meta-parameters like 'market sentiment' is very crucial [Taylor S.J (2005) ; Rossi PE (1996)]. We have discussed this aspect in detail, in this particular section.

Volatility and stock market trends are intertwined because volatility reflects the degree of price fluctuations, and these fluctuations can influence investor behaviour, trading strategies, and perceptions of risk, which in turn can impact broader market trends.

The implied volatility (VIX) is a measure of market expectations for future volatility. It's often referred to as the "fear index" because it tends to rise when investors anticipate increased uncertainty and potential price swings. The VIX is calculated using option prices and reflects the collective sentiment of market participants regarding future volatility.

There is an asymmetric relationship between VIX and stock market returns [Chandra and Thenmozhi (2015); Sarwar (2012)]. VIX indicates how investors and equity traders are thinking about the overall health of the current economy and near-term market fluctuations based on the options market activities. Prediction of VIX itself can be crucial for predicting stock market trends [Ruan et al. (2018)].

Realized volatility, on the other hand, is a measure of actual price fluctuations that have occurred in the past. It's calculated based on historical price movements over a specific period. [Taylor S.J (2005)]

The spread between implied volatility (VIX) and realized volatility can offer insights into the market's perception of future volatility compared to the actual volatility that has been observed. [Taylor S.J (2005); Rossi PE (1996); Natenberg S (2014)]

The connection between 'market sentiments' and volatility is symbiotic. [Thaler R.H (2017)]. Das A.R [2020] has established some good connections between market sentiment and SP500 price movements.

However, an analysis which would establish clear connections between market sentiments and volatility, was not very evident in recent literature. Hence, we chose to perform the same in this part of the work.

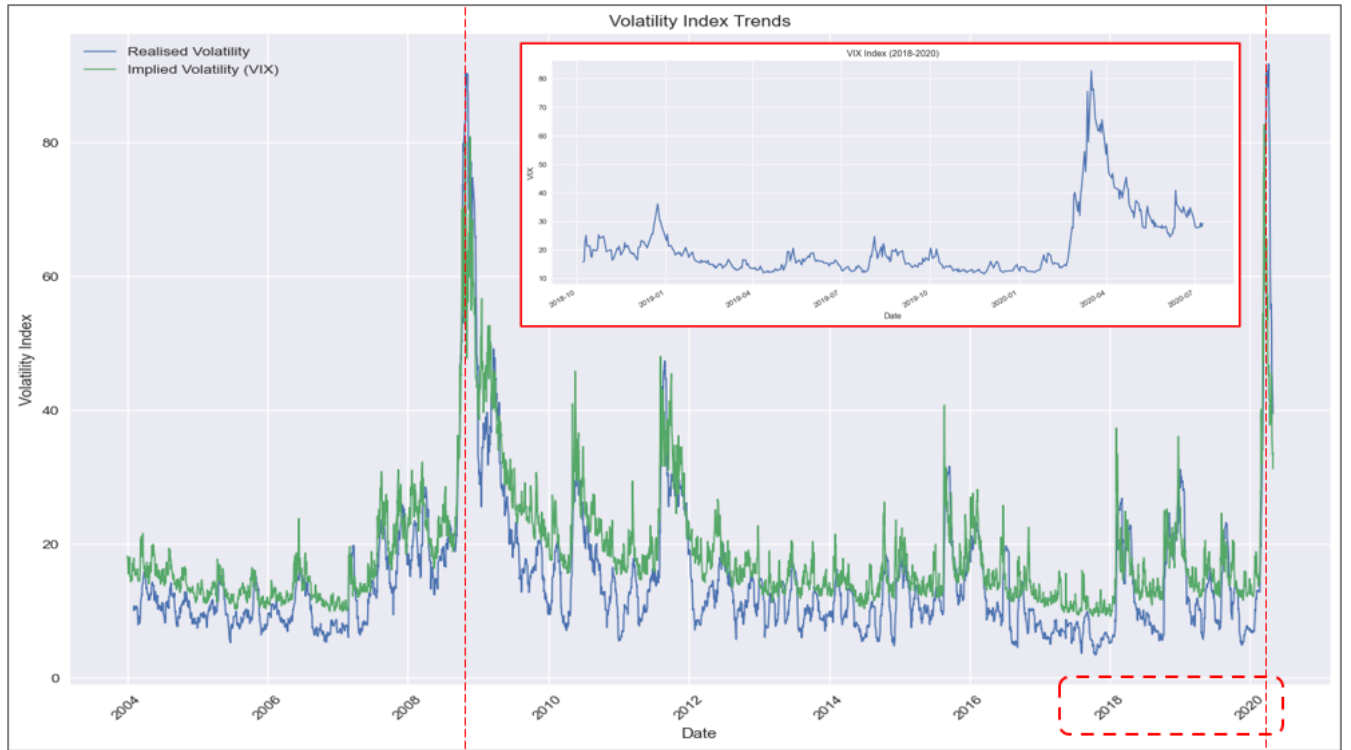


Figure 14: Volatility Trends

We first chose a small part of the window (2018 - 2020), for this part of the work, as shown in figure 14. We can see the spread between realised and implied volatility (VIX) for the entire period of study (2004 - 2020) and the VIX for window of study in this part (2018-2020), in the subset window, in figure 14.

If we carefully observe the 'low volatile' periods in figure 14, we see the realised volatility to be lower than the implied volatility. This normally indicates a bullish perspective. On the other hand, the 'high volatile' periods have 1] significant volatility peaks and 2] VIX lower than realised volatility. This indicates a bearish perspective. Hence, this re-affirms our choice of target variable that, a positive RV-IV spread indicates a bear move and otherwise.

Figure 15 shows the comparative views of volatility spread vs actual values. The zones where the red dotted line is above level 0 (gray horizontal line), are the ones with bearish perspectives.

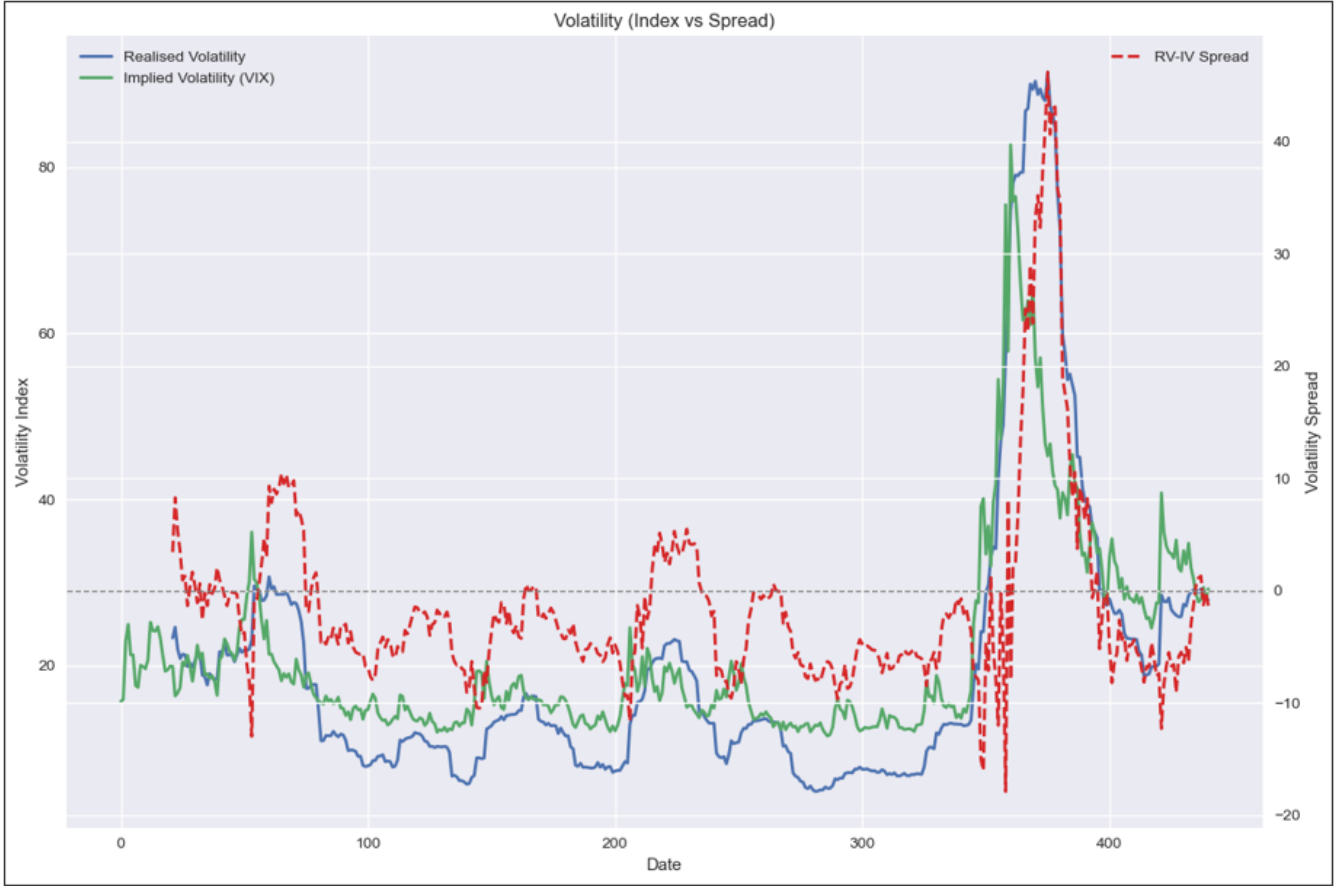


Figure 15: Volatility Spread vs Index

8.1 Sentiment Analysis

Das A.R [2020] has performed some good analysis on SP500 prices vs sentiment data. The article explores trading strategies using Reddit sentiment data from the wallstreetbets subreddit. It employs VADER sentiment analysis for comments' positive/negative sentiment and keyword analysis for bullish/bearish sentiment in submission titles. The author applies Fourier transforms for data smoothing and normalization, presenting two trading strategies: one based on SPY prices and sentiment spread, and another using LSTM models for sentiment-driven price prediction.

We have consumed the sentiment data from the above mentioned article and have also adopted the normalization methods detailed in this article for this particular part of our work. We apply the sentiment scores on volatility spreads (as against prices), to be able to derive insights on market moves.

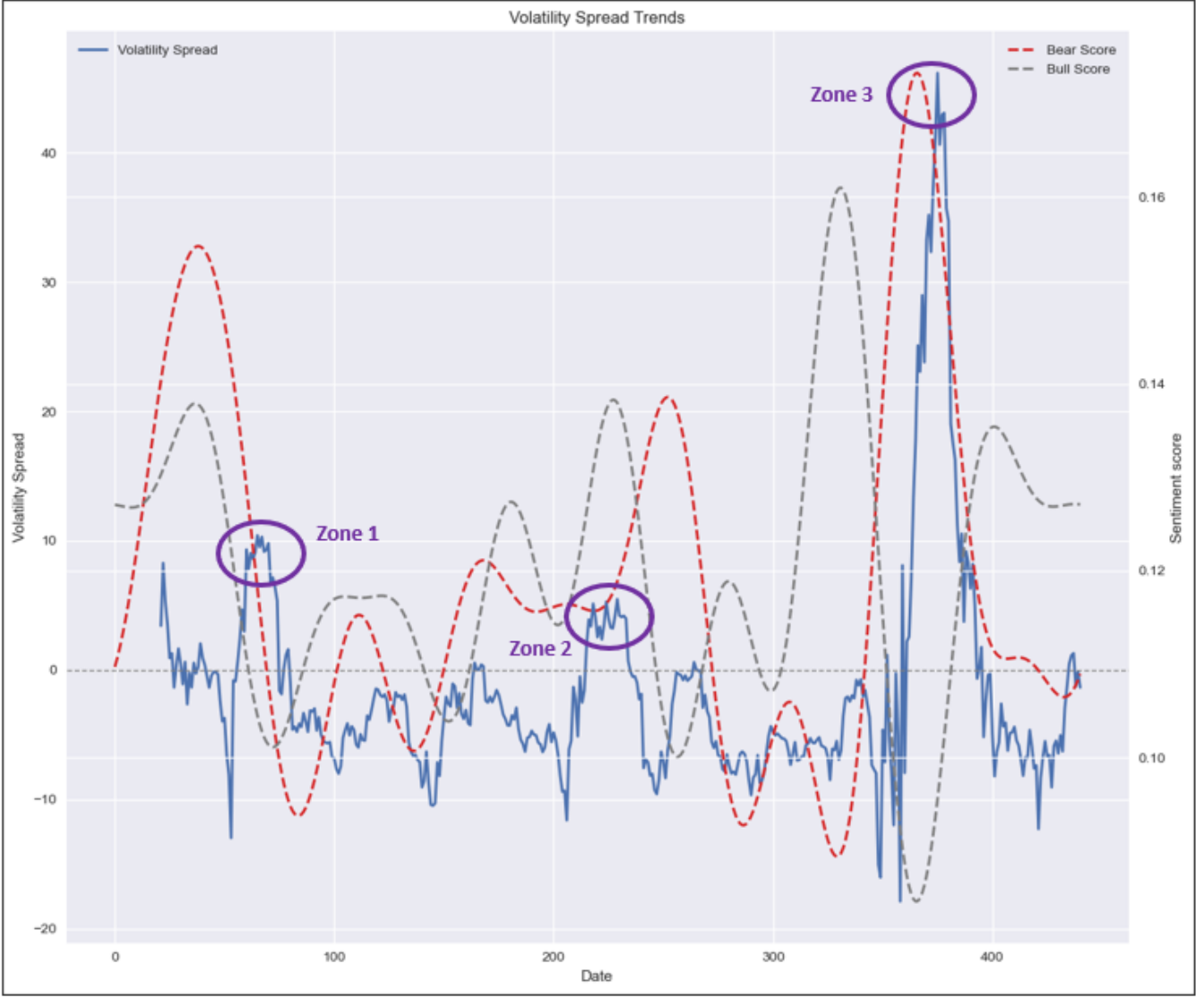


Figure 16: Volatility Spread vs Sentiment

In figure 16, the purple zones are key points to notice, as they are the points where we have positive RV-IV peak. However, the interesting point to note here is that, the three points shown are unique by itself. In zone 3, the high RV-IV spread can be directly related to a high bear score (red dotted line). In Zone 2, the bear score is increasing only after the RV-IV peak is established and in zone 1, the RV-IV peak is arriving, only after a clear 'bear score peak'. However, in all three zones, the RV-IV spreads are closely related to a 'bear' move.

Now, this re-substantiates that either market sentiments can impact volatility (Zone 1) or vice verse (Zone 2). The key connecting factor between sentiment and volatility could be the VIX index. The connection between the VIX and market sentiments is complex and intertwined. While heightened market sentiments can contribute to increased volatility and impact the VIX, an elevated VIX can also influence investor emotions and sentiments. Market conditions, news, and events can trigger changes in both sentiments and the VIX, and their dynamic relationship creates a cycle of influence and feedback in the financial markets.

9 Portfolio Construction and Investment Strategies

[Lin et al. (2022)] have proposed a methodology for stock selection based on a special ratio which is forward PE divided by trailing PE. This ratio (as they propose) can better evaluate the growth of individual stocks. In their work, they performed a price prediction analysis using LSTM, to show that stocks that have low

Company	SP500 Allocation(in %)	PEG Ratio	Score 1	Sector	Sector Weight(in %)	Score 2
AAPL	7.70	2.54	3.03	Info. Tech	28.3	8.58
MSFT	6.89	3.08	2.24	Info. Tech	28.3	6.33
META	1.75	1.12	1.56	Info. Tech	28.3	4.42
GOOGL	1.94	1.52	1.28	Info. Tech	28.3	3.61
AMZN	3.11	2.45	1.27	Info. Tech	28.3	3.59
GOOG	1.68	1.53	1.10	Info. Tech	28.3	3.11
NVDA	2.82	5.45	0.52	Info. Tech	28.3	1.46
UNH	1.18	1.48	0.80	Healthcare	13.4	1.07
XOM	1.13	0.58	1.95	Energy	4.1	0.80
V	1.05	1.84	0.57	Financials	12.4	0.71
Berk H	1.65	3	0.55	Financials	12.4	0.68
JPM	1.13	2.06	0.55	Financials	12.4	0.68
LLY	0.99	2.12	0.47	Healthcare	13.4	0.63
MA	0.89	1.88	0.47	Financials	12.4	0.59
JNJ	1.14	1.88	0.40	Healthcare	13.4	0.53
MRK	0.77	2.1	0.37	Healthcare	13.4	0.49
AVGO	0.96	1.72	0.56	Comm.Serv	8.4	0.47
HD	0.83	2.07	0.40	Cons. Dsc	10.7	0.43
CVX	0.72	0.83	0.87	Energy	4.1	0.36
ABBV	0.66	2.62	0.25	Healthcare	13.4	0.34
PEP	0.70	3.1	0.23	Cons. Dsc	10.7	0.24
TSLA	2.02	3.6	0.56	Energy	4.1	0.23
KO	0.64	3.53	0.18	Cons. Dsc	10.7	0.19
PG	0.97	4.18	0.23	Cons. Stp	6.7	0.16
COST	0.65	4.65	0.14	Industrials	8.5	0.12

Table 6: Portfolio Optimisation Scorecard

PE ratios had strong growth. But in this work, many of the stocks were not having proximity to the tangency portfolio and this work did not opine clearly on the optimal portfolio. However, this is a good piece of work which motivated us to perform the analysis detailed in this section.

When considering stocks from standard indices like SP500, considering aspects like portfolio weights, diversification is crucial for constructing an optimal portfolio.

A balanced methodology or proposal for stock selection based on the the mentioned aspects, was not very evident in recent literature.

Hence, this section proposes a novel methodology for stock selection based on a balanced scorecard.

9.1 Methodology Description

The description of the methodology is as follows. At the outset, the top 25 stocks from SP500 were selected [source : Investopedia]. The top 25 ranking was based on portfolio allocation (in %). For eg, in table 6, AAPL (Apple) stock covers 7.7 % of SP500. This is how the second column in table 6 needs to be interpreted.

Secondly, we considered an important parameter called the PEG (Price to earnings to growth) ratio. The PEG ratio, which stands for Price/Earnings-to-Growth ratio, is a financial metric used to assess the valuation of a company's stock relative to its expected earnings growth. It takes into account both the company's price-to-earnings (P/E) ratio and its projected earnings growth rate. The formula for calculating the PEG ratio is:

$$PEG\ Ratio = P/E\ Ratio / Earnings\ Growth\ Rate$$

The PEG ratio combines these two metrics to provide a more comprehensive view of a company's valuation. A PEG ratio below 1 might suggest that the stock is undervalued relative to its growth potential, which could indicate the stock having higher growth potential. This is not the same as the P/E ratios used in [Lin et al. (2022)] since the growth rate here can be based on historical earnings growth, analyst estimates, or other relevant factors. Hence, PEG ratio could be more holistic. At this point, we calculate a score as below :

$$\text{Score 1} = \text{SP500 Allocation} / \text{PEG Ratio}$$

A higher 'score 1' is indicative of a better stock, since it would have a higher allocation and a lower PEG ratio. Score 1 for all the 25 stocks is given in col 4, in table 6. We sourced the PEG ratios for each of the tickers from NASDAQ website.

Another important factor for a balanced portfolio is diversification. the SP500 is diversified across 11 major sectors (given in decreasing order of sector weights) viz. 1] Information Tech ; 2] Healthcare 3] Financials 4] Consumer Discretionary 5] Industrials 6] Communication Services 7] Consumer Staples 8] Energy 9] Utilities 10] Materials and 11] Real Estate.

Hence, we labelled each of the 25 stocks, as per their respective sectors and assigned the respective sectors weights (Table 6 : col 5 and 6). [Source of SP500 sectors and sector weights: Investopedia].

Finally, we arrived at 'Score 2', which is the individual 'Score 1' of each stock multiplied by its corresponding sector weight :

$$\text{Score 2} = \text{Score 1} * \text{Sector Weight}$$

Hence, a high 'score 2' is indicative of a balanced stock, which would have good allocation, better growth and would be diversified in line with the index's sector weights. This was the final metric with which we prepared the detailed scorecard that is shown in table 6. We finally chose the 10 stocks that is highlighted in figure 17, to construct our portfolio. Our selection is based on considering all aspects w.r.t 'score 2'. Hence, we just did not choose the 'top 10' stocks from table 2, but chose a balanced set of stocks to make our portfolio as diverse as possible.

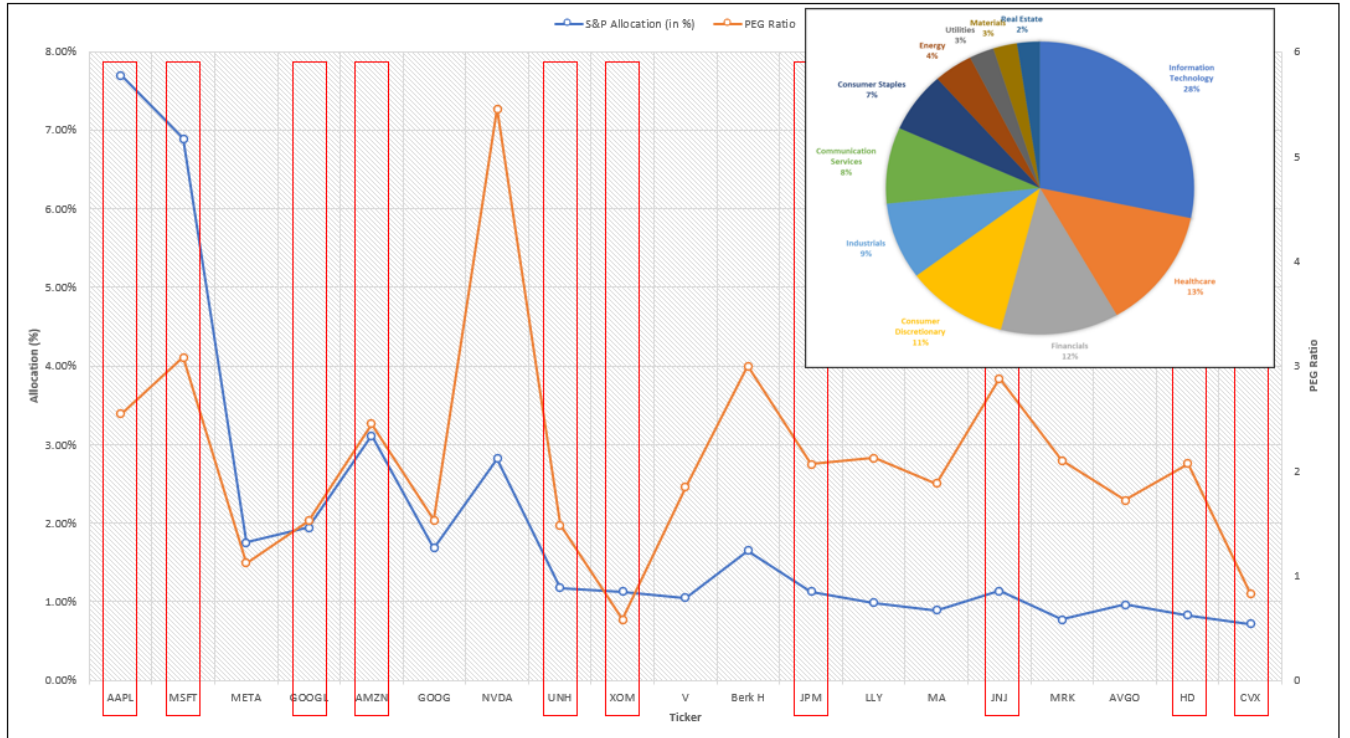


Figure 17: Portfolio Selection

9.2 Portfolio Optimization

At the outset, before the portfolio construction exercise, we performed a stock price prediction experiment, on a individual stock (AAPL). We used our single layer LSTM model for this experiment, for us to be able to reaffirm the fact that our LSTM framework would assist well for design of investment strategies.

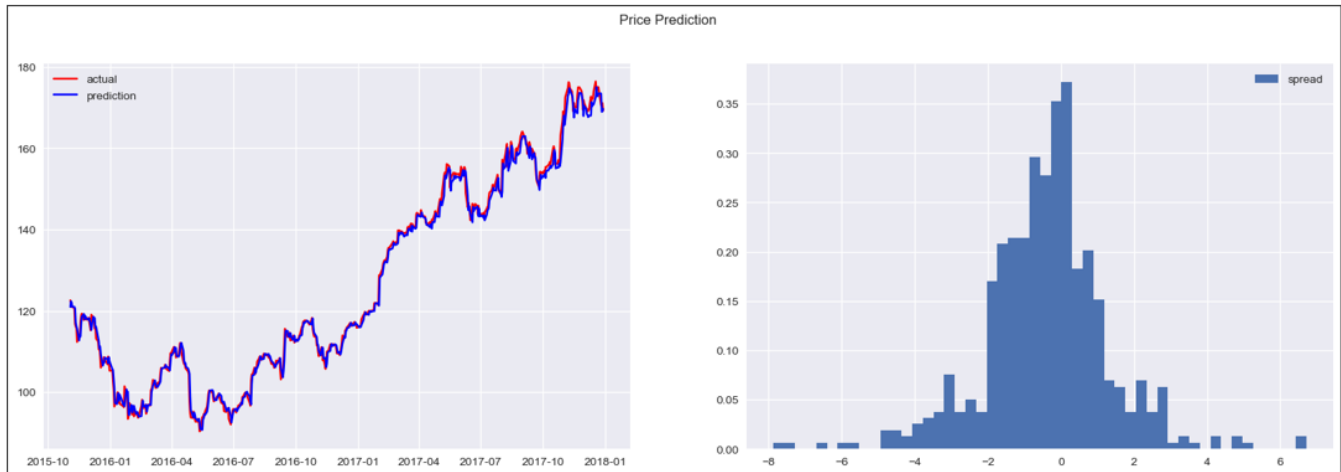


Figure 18: AAPL Price Prediction using LSTM

We consistently obtained a prediction accuracy of 0.99 (results evident from figure 18).

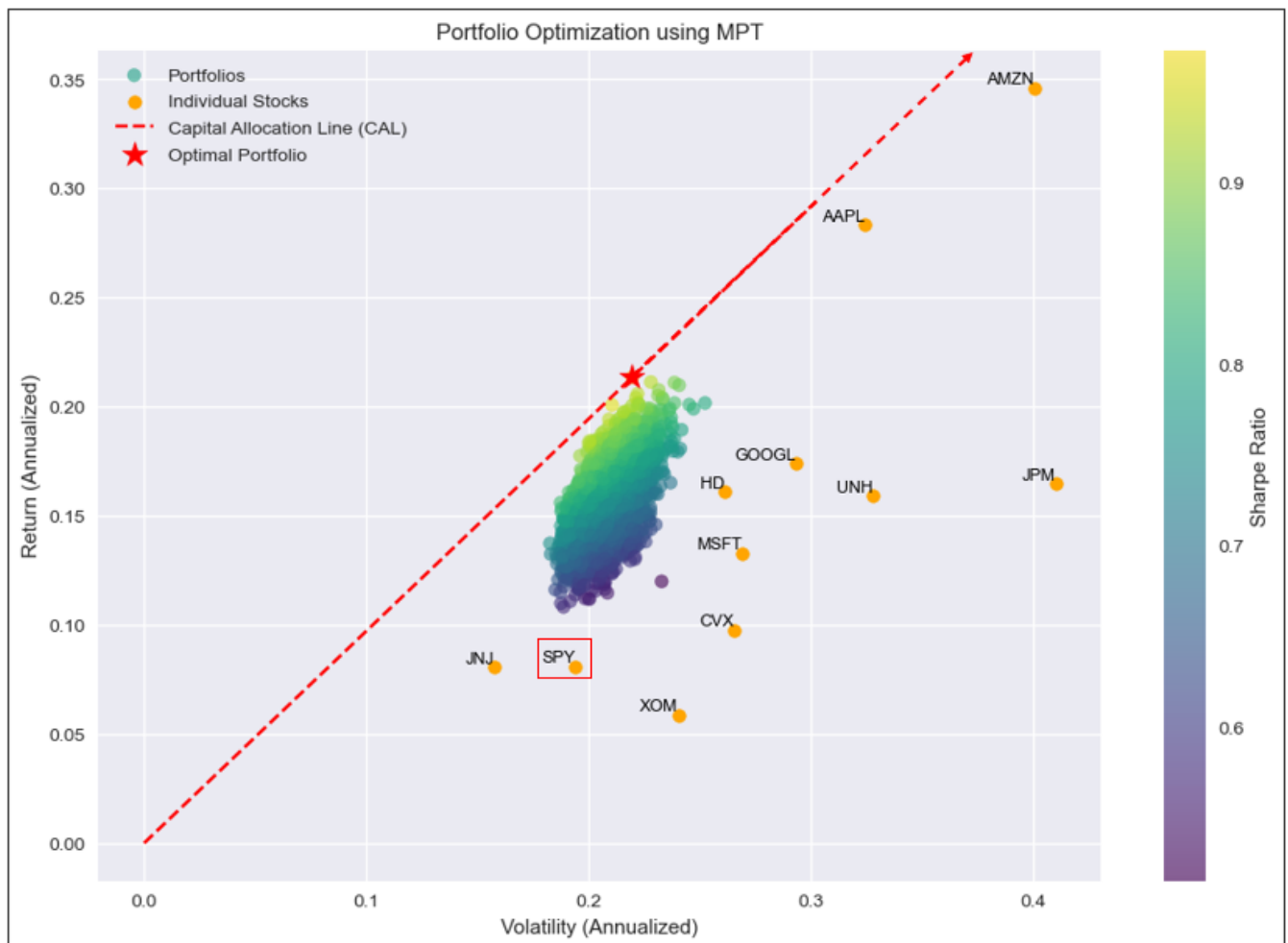


Figure 19: Portfolio Optimization using MPT

We the proceed to construct the portfolio with the selected 10 stocks, using modern portfolio theory (MPT).

We obtained promising results, as shown in figure 19. Set at a risk free rate of 0 %, the red dotted line in figure 19, shows the capital allocation line (CAL) and the red star shows our optimal portfolio. The individual stocks are market in orange and the SP500 index is marked in a red rectangle for reference.

The placement of our optimal portfolio is indicative of the fact that it is well balanced w.r.t an optimal return and annualised volatility. In fact, we could possibly argue that the optimal portfolio could perform better the SP500 index itself, as its having a much higher return for a marginal increase in volatility. Also the optimal portfolio outperforms other individual stocks w.r.t return vs volatility comparison. Another key observation is that the best stock of SP500 (AAPL) is the one, that is having the closed proximity to the CAL (however with a higher volatility).

Hence, in this section, we have offered a novel methodology for portfolio construction using stable stocks from standard indices. This methodology can be applied across various other standard indices.

10 Trading Strategy

A trading strategy was designed using pyfolio package. The strategy exhibited a promising performance over the 45-month evaluation period, delivering an annualized return of 12.867 % with cumulative returns of 57.597 %. While the strategy demonstrated a favorable risk-adjusted performance with a Sharpe ratio of 0.78 and a Sortino ratio of 1.10, its annual volatility of 17.599 % and maximum drawdown of -20.837 % indicate a moderate level of risk. The Omega ratio of 1.15 suggests a generally positive distribution of gains and losses. However, the presence of negative skew (-0.24) and higher kurtosis (4.76) hints at some non-normality and potential for extreme returns. The strategy's daily value at risk stands at -2.163 % , indicating a conservative estimate of its maximum daily loss. These results emphasize a balance between favorable returns and associated risks, encouraging a thorough risk assessment aligned with individual risk tolerance and investment objectives.

Table 7: Trading Strategy Results

Metric	Value
Start date	2016-03-30
End date	2019-12-31
Total months	45
Backtest	
Annual return	12.867%
Cumulative returns	57.597%
Annual volatility	17.599%
Sharpe ratio	0.78
Calmar ratio	0.62
Stability	0.88
Max drawdown	-20.837%
Omega ratio	1.15
Sortino ratio	1.10
Skew	-0.24
Kurtosis	4.76
Tail ratio	0.97
Daily value at risk	-2.163%

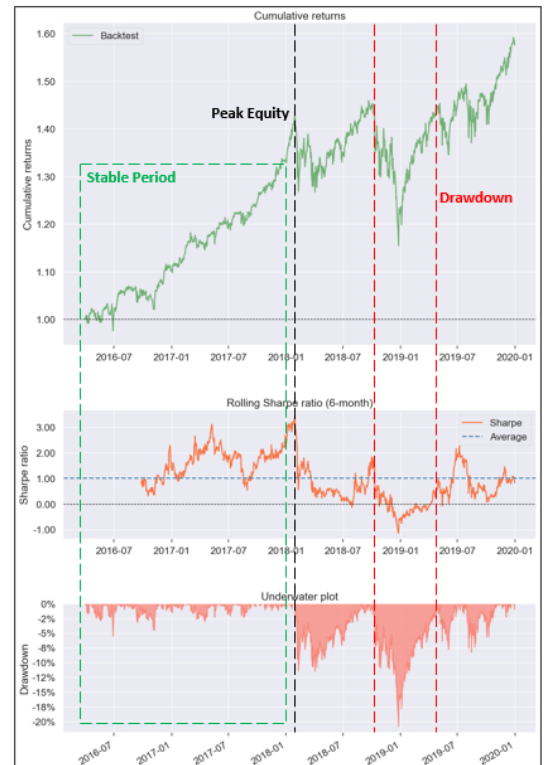


Figure 20: Trading Strategy

11 Conclusion and Future Work

Key highlights of the work

1. A novel methodology for stock selection from SP500 indices has been proposed through the utilization of a balanced score. This score incorporates the portfolio allocation of each stock, PEG ratio, and the diversification factor via sector weights. The selection of 10 stocks out of the top 25 SP500 stocks was executed using the proposed methodology, revealing that a portfolio of 10 stocks surpasses the performance of all other stocks (including the SP500 index itself). This accomplishment was substantiated by conducting a portfolio optimization exercise using MPT.

2. A comprehensive deep learning framework has been designed for the prediction of stock price trends, with a focus on using volatility spreads as the target variable. A relatively simple dataset comprising solely the SP500 price and VIX (Implied Volatility) was employed. The target variable encompasses the spread between realized and implied volatility (RV-IV). Based on this foundation, a comprehensive deep learning framework using LSTMs has been crafted, poised for optimal performance in predicting stock trends. A meticulous hyperparameter tuning exercise involving diverse algorithms and parameters was carried out, with subsequent comparisons against a base classifier (random forest). While the LSTM-based models achieve an average accuracy of approximately 85-90 %, the base classifier demonstrates superior performance, boasting a classification accuracy of 93% and a balanced classification report. Therefore, the recommendation is to opt for simpler datasets and traditional classifiers like random forest or SVMs for predicting stock trends.

3. Additionally, a prediction exercise was undertaken to forecast the implied volatility (VIX) and the SP500 price using a single-layer LSTM. Impressively, commendable results were attained, solidifying the recommendation for utilizing LSTM-based models in stock value prediction. The intricacies of the deep learning framework are encompassed within points 2 and 3.

4. An analysis delving into the connection between volatility and market sentiments was carried out, unraveling the symbiotic nature of these two elements. Detailed insights were presented, highlighting how volatility and market sentiments intertwine and how investors can leverage such analyses for informed decision-making. Concluding this exploration, a bridge was forged, connecting price prediction analysis with trading strategies.

Scope for future work

Looking ahead, there lies an avenue to explore the refinement of models tailored for volatility-based predictions, building upon the foundations laid earlier. Aiming for more optimal outcomes, the emphasis can shift towards crafting models that excel in capturing volatility trends. Additionally, the spotlight could be directed towards optimal model tuning, with the overarching goal of mitigating overfitting risks. This would involve a careful interplay of parameter selection and algorithm fine-tuning, ensuring that the predictive prowess of the models extends to unseen data and real-world scenarios. As we peer into the horizon of future research, the trajectory is marked by the pursuit of even greater precision and reliability in volatility-based predictions.

12 Ethics and Implications

In this study, we utilized publicly available data without alteration except as detailed in the paper. While the outcomes offer supplementary insights for investment choices, it's cautioned that decisions should not rest solely on this research. Investors are advised to conduct due diligence, accounting for their risk tolerance amidst varying market conditions. Effective forecasting requires considering both the model's findings and the volatile nature of markets, particularly in scenarios like geopolitical tensions, supply chain disruptions, wars, pandemics, and others. By aligning market behavior analysis with the model's outcomes, profit potential can be harnessed. The aspiration to precisely predict stock prices and anticipate returns is shared by traders, investors, and portfolio managers. This study underscores the potential of LSTM neural networks to outline the scope of uncertainty in stock price predictions. Stakeholders can incorporate these findings in their decision-making process, while researchers can apply the model to advance their understanding of sequential data modelling.

References

Electives Reviewed: Advanced Machine Learning I and II ; Algorithmic Trading I ; Behavioural Finance for Quant

Other references are as below :

1. Anderson K, Brooks C. The long-term price-earnings ratio. *Journal of Business Finance and Accounting*, 2006, 33(7-8): 1063-1086.
2. Bao, W., Yue, J., and Rao, Y. (2017). A deep learning framework for financial time series using stacked autoencoders and long-short term memory. *PLoS One*, 12(7), 1–24.
3. Bhandari H.N, Rimal B, Pokhrel N.R (2022).Predicting stock market index using LSTM. *Machine Learning with Applications*. <https://doi.org/10.1016/j.mlwa.2022.100320>
4. Bishop C.M. *Pattern Recognition and Machine Learning* (2006). Springer. ISBN 978-0-387-31073-2.
5. Breiman. L. *Random Forests* (2001). *Machine Learning*. <https://link.springer.com/article/10.1023/>
6. Chandra, A., and Thenmozhi, M. (2015). On asymmetric relationship of India volatility index (India VIX) with stock market return and risk management. *Decision*, 42(1), 33–55.
7. Chen, K., Zhou, Y., and Dai, F. (2015). A LSTM-based method for stock returns prediction. A case study of China stock market. In *2015 IEEE international conference on big data (Big Data)* (pp. 2823–2824).
8. Das. A.R . *Sentiment Analysis for Trading with Reddit Text Data* (2020).[medium.com/sentiment-analysis-for-trading-with-reddit-text-data]
9. Easton PD. PE ratios, PEG ratios, and estimating the implied expected rate of return on equity capital. *The accounting review*, 2004, 79(1): 73-95.
10. Fischer T, Krauss C. Deep learning with long short-term memory networks for financial market predictions. *Eur J Oper Res*. 2018;270(2):654–69. <https://doi.org/10.1016/j.ejor.2017.11.054>.
11. Gao, P., Zhang, R., and Yang, X. (2020). The application of stock index price prediction with neural network. *Mathematical and Computational Applications*, 25(3), 53.
12. Gao, Y., and Yan, X. (2019). Stock price prediction based on deep neural networks. *Neural Computing and Applications*, 32, 1609–1628.
13. Hochreiter S, Schmidhuber J. Long Short-Term Memory. *Neural Computation* (1997) 9 (8): 1735–1780.
14. Huang CL, Tsai CY. A hybrid SOFM-SVR with a filter-based feature selection for stock market forecasting. *Expert Syst Appl*. 2009;36(2 PART 1):1529–39. <https://doi.org/10.1016/j.eswa.2007.11.062>.
15. Idrees SM, Alam MA, Agarwal P. A prediction approach for stock market volatility based on time series data. *IEEE Access*. 2019;7:17287–98. <https://doi.org/10.1109/ACCESS.2019.2895252>.
16. Karmiani, D., Kazi, R., Nambisan, A., Shah, A., and Kamble, V. (2019). Comparison of predictive algorithms: Backpropagation, SVM, LSTM and Kalman filter for stock market. In *2019 amity international conference on artificial intelligence (AICAI)* (pp. 228–234).
17. Kara Y, Acar Boyacioglu M, Baykan ÖK. Predicting direction of stock price index movement using artificial neural networks and support vector machines: the sample of the Istanbul Stock Exchange. *Expert Syst Appl*. 2011;38(5):5311–9. <https://doi.org/10.1016/j.eswa.2010.10.027>.
18. Kim K, Han I. Genetic algorithms approach to feature discretization in artificial neural networks for the prediction of stock price index. *Expert Syst Appl*. 2000;19:125–32. [https://doi.org/10.1016/S0957-4174\(00\)00027-0](https://doi.org/10.1016/S0957-4174(00)00027-0).
19. Lanbouri, Z., and Achhab, S. (2020). Stock market prediction on high frequency data using long-short term memory. *Procedia Computer Science*, 175, 603–608.
20. Lei L. Wavelet neural network prediction method of stock price trend based on rough set attribute reduction. *Appl Soft Comput J*. 2018;62:923–32. <https://doi.org/10.1016/j.asoc.2017.09.029>.
21. Lee MC. Using support vector machine with a hybrid feature selection method to the stock trend prediction. *Expert Syst Appl*. 2009;36(8):10896–904. <https://doi.org/10.1016/j.eswa.2009.02.038>.
22. Li L, Jamieson K, DeSalvo G, Rostamizadeh A, Talwalkar A. Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization (2018). *Machine Learning*. <https://arxiv.org/abs/1603.06560>.
23. Natenberg . S. *Option Volatility and Pricing*. (2014) McGraw Hill. ISBN: 978-0071818773.

24. Nekoeigachkanloo H, Ghogh B, Pasand AS, Crowley M. Artificial counselor system for stock investment. 2019. ArXiv Preprint arXiv:1903.00955.
25. Ni LP, Ni ZW, Gao YZ. Stock trend prediction based on fractal feature selection and support vector machine. *Expert Syst Appl.* 2011;38(5):5569–76. <https://doi.org/10.1016/j.eswa.2010.10.079>.
26. Olah C . Understanding LSTM Networks. (2015). [colah.github.io/posts/Understanding-LSTMs/]
27. Piramuthu S. Evaluating feature selection methods for learning in data mining applications. *Eur J Oper Res.* 2004;156(2):483–94. [https://doi.org/10.1016/S0377-2217\(02\)00911-6](https://doi.org/10.1016/S0377-2217(02)00911-6).
28. Qiu M, Song Y. Predicting the direction of stock market index movement using an optimized artificial neural network model. *PLoS ONE.* 2016;11(5):e0155133.
29. Qiu, J., Wang, B., and Zhou, C. (2020). Forecasting stock prices with long-short term memory neural network based on attention mechanism. *PLoS One*, 15(1), 1–15.
30. Roondiwala, M., Patel, H., and Varma, S. (2017). Predicting stock prices using LSTM. *International Journal of Science and Research (IJSR)*, 6(4), 1754–1756.
31. Rossi, P.E. “Modelling stock market volatility : Bridging the gap to continuous time” (1996). Academic Press. ISBN: 0-12-598275-5
32. Ruan, L. (2018). Research on sustainable development of the stock market based on VIX index. *Sustainability*, 10(11), 4113.
33. Rudnicki Miron B. Kursa Aleksander Jankowski. “Boruta – A System for Feature Selection”. ICM, University of Warsaw (2010).
34. Sarwar, G. (2012). Is VIX an investor fear gauge in BRIC equity markets? *Journal of Multinational Financial Management*, 22(3), 55–65.
35. Shen, J and Shafiq, O M. (2020). Short term stock market price trend prediction using a comprehensive deep learning system. *J. Big Data* (2020) 7:66 <https://doi.org/10.1186/s40537-020-00333-6>.
36. Shwartz S.S ; David S.B. Understanding Machine Learning: From Theory to Algorithms. (2014) Cambridge University Press. ISBN 978-1107057135.
37. Snoek J, Larochelle H, and Adams R.P. Practical Bayesian Optimization of Machine Learning Algorithms. (2012). *Machine Learning*. <https://arxiv.org/abs/1206.2944>.
38. Taylor S.J. “Asset Price Dynamics, Volatility, and Prediction”. (2005). Princeton university press. ISBN: 978-0-691-13479-6.
39. Thakur M, Kumar D. A hybrid financial trading support system using multi-category classifiers and random forest. *Appl Soft Comput J.* 2018;67:337–49. <https://doi.org/10.1016/j.asoc.2018.03.006>.
40. Thaler R.H. Misbehaving: The Making of Behavioral Economics. *The Review of Austrian Economics* volume 30, pages137–141 (2017). <https://link.springer.com/article/10.1007/s11138-015-0330-z>
41. Tsai CF, Hsiao YC. Combining multiple feature selection methods for stock prediction: union, intersection, and multiintersection approaches. *Decis Support Syst.* 2010;50(1):258–69.
42. Truong C. Value investing using price earnings ratio in New Zealand. *University of Auckland. Business Review*, 2009, 11(1): 26.
43. Wilmott. P. *Machine Learning : An Applied Mathematics Introduction.* (2020); Panda Ohana Publishing. ISBN: 978-1-9160816-0-4.
44. Witold R. Rudnicki Miron B. Kursa Aleksander Jankowski. “Boruta – A System for Feature Selection”. ICM, University of Warsaw (2010).
45. Yadav, A., Jha, C. K., and Sharan, A. (2020). Optimizing LSTM for time series prediction in Indian stock market. *Procedia Computer Science*, 167, 2091–2100.
46. Yu, P., and Yan, X. (2019). Stock price prediction based on deep neural networks. *Neural Computing and Applications*, 32, 1609–1628.
47. Z. Lin, F. Tian and W. Zhang. (2022). Evaluation and Analysis of an LSTM and GRU Based Stock Investment Strategy. Open access. <https://doi.org/10.2991/978-94-6463-052-7>.

For content related to Machine Learning : Relevant Lectures and Labs from Modules 4 and 5 were referred.

For content related to Portfolio Optimization : Relevant lectures from Module 2, were referred.